MACHINE LEARNING / MODULE 1

Supervised learning foundations

Regression, classification and honest evaluation for energy data

Full day · 09:00–17:00

6 parts · 3 labs

Python · scikit-learn

Machine learning for energy applications

Today's plan

09:00	Welcome and framing supervised problems	Lecture
09:30	Data, EDA and preprocessing	Lecture
10:10	Break	Break
10:25	Splits, cross-validation and leakage	Lecture
11:00	Linear models and regularization	Lecture
11:50	Lab 1: regression and regularization	Lab
12:40	Lunch	Lunch
13:30	Classification models	Lecture
14:30	Lab 2: classifiers and a facies split design	Lab
15:25	Break	Break
15:40	Evaluating and selecting models	Lecture
16:15	Lab 3: a tuned pipeline and a locked test	Lab
16:45	Recap and exit check	Discussion

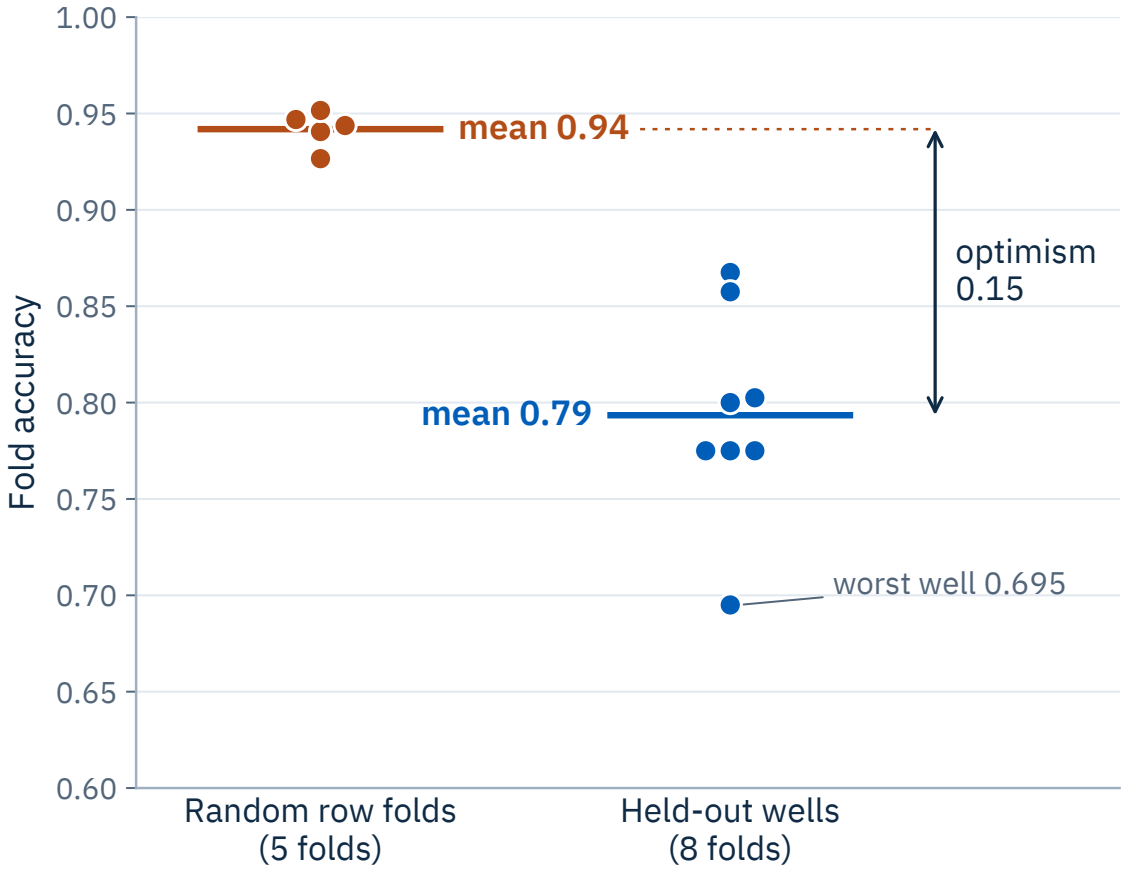
What you will be able to do

By the end of the day you will be able to:

1. Write a prediction contract: target, unit, prediction time, inputs and metric.
2. Audit energy data for null codes, unit mixes, depth shifts and patterned gaps.
3. Choose a split and cross-validation scheme that matches deployment.
4. Fit and regularize linear models and diagnose them with residuals and learning curves.
5. Explain the boundaries of logistic regression, KNN, SVM and decision trees.
6. Evaluate classifiers per class and tune them inside grouped cross-validation.

Same model, same wells, two very different scores

- A 1-nearest-neighbour facies model: 8 synthetic wells, 5 logs
- Random row folds: mean accuracy 0.94
- Held-out wells: mean 0.79; the worst well 0.695
- Which number belongs in the report to the asset team?



Computed on a synthetic 8-well dataset (seed 7); standardized 1-NN classifier. Not field performance.

PART 1 OF 6

01

Framing supervised problems

A model is only as useful as the question it answers.

- Prediction contracts
- Task types and shared vocabulary
- The end-to-end workflow and the energy landscape
- Baselines that make improvement measurable

PRACTICE

3 min · Pairs

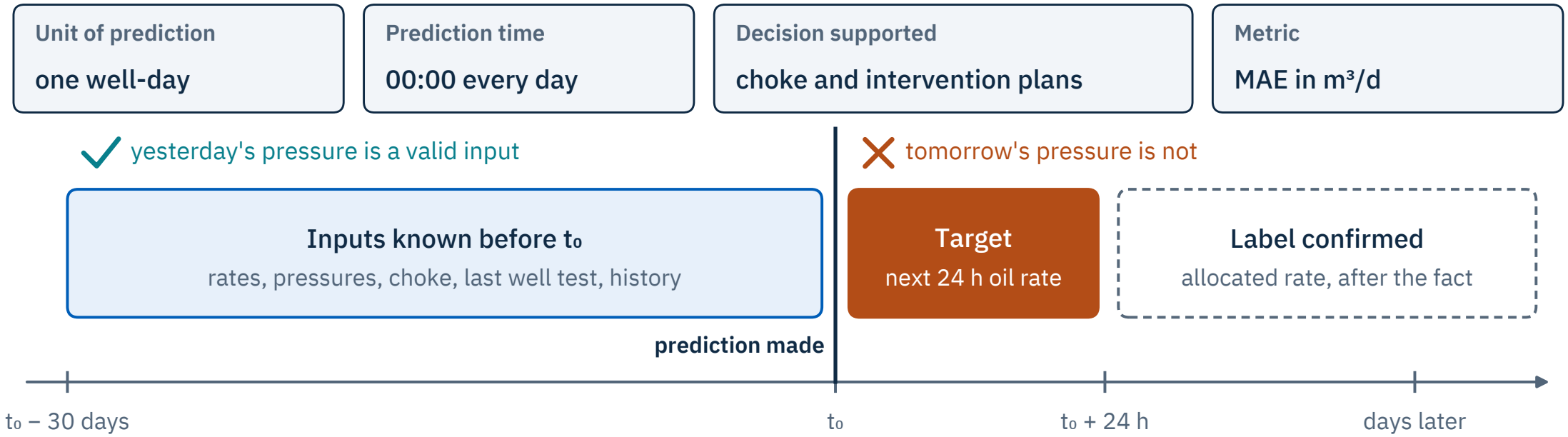
A prediction needs a contract

An operator wants to predict poor production.

1. What exactly is the target, and over what future interval?
2. Is one example a depth sample, a well, or a well-day?
3. Which measurements exist at the moment of prediction?

A contract fixes target, timing, inputs and action

One worked answer: next-day oil rate per well, predicted at midnight.



Schematic prediction contract. The same template applies to any energy forecast or classification.

Supervised learning learns from labelled examples

Regression

- Numeric target
- Porosity, next-day rate, hourly load

Classification

- Class label
- Facies, trip or no trip, fault type

Unsupervised

- No target: structure only
- Groups of production patterns (Modules 3–4)

Reinforcement

- Actions and delayed rewards
- Control policies; background only

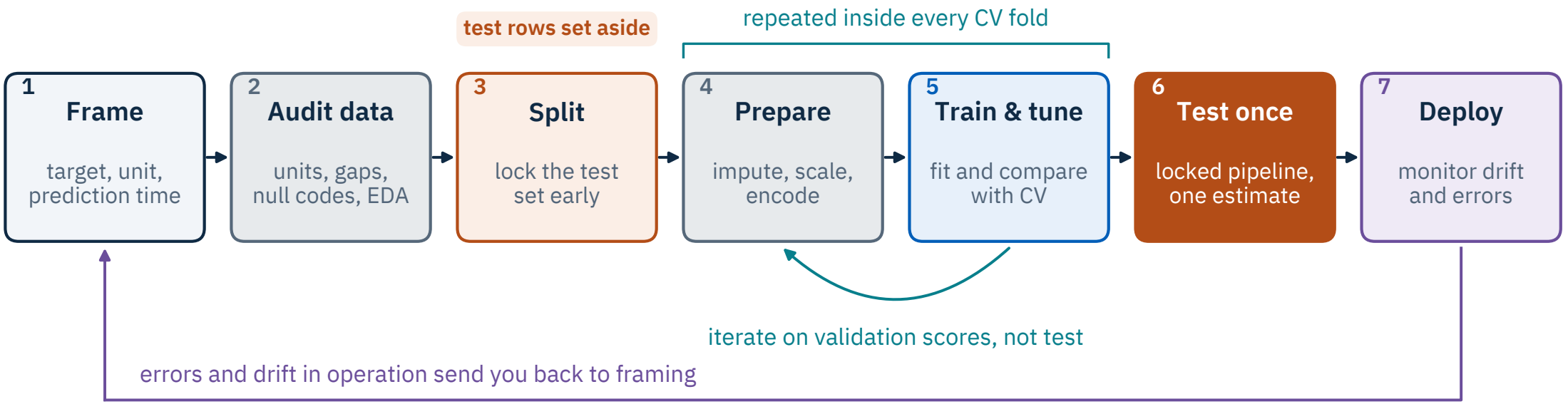
◆ Supervised means every training row carries a known answer: a number or a class.

The vocabulary we will use all week

Term	Meaning	Energy example
Feature	An input known at prediction time	GR and RHOB at one depth
Target	The value or label to predict	Core porosity, facies
Observation / group	One row / rows that belong together	Depth sample / well
Training, validation, test	Fit / choose / check once	6, 2 and 2 wells
Baseline	The simplest defensible reference	Training mean, persistence
Leakage	Information the model would not have	Tomorrow's pressure as input

The workflow is a loop with the test set locked early

Steps 4–5 repeat inside cross-validation; step 6 happens once per locked pipeline.



Schematic supervised-learning workflow. Colours: navy framing, grey data steps, blue model fitting, orange test data, purple operation.

Supervised problems span the energy value chain

Subsurface	Drilling	Production	Facilities	Power
Facies from logs — per depth sample — C	Rate of penetration — per metre drilled — R	Next-day oil rate — per well-day — R	Corrosion rate — per pipe segment — R	Day-ahead load — per hour — R
Porosity from logs — per depth sample — R	Stuck-pipe warning — per rig hour — C	ESP trip in 7 days — per pump-day — C	Leak-alarm triage — per alarm — C	Wind-farm output — per 10 minutes — R
Missing sonic log — per depth sample — R	Lithology at the bit — per depth sample — C	Gas-lock detection — per time window — C	Compressor fault — per trip event — C	Transformer fault — per gas-analysis sample — C

R regression: numeric target C classification: class label

Schematic overview of typical problem formulations; examples only, no performance implied. R = regression, C = classification.

Baselines make improvement measurable

Regression

- Training-target mean or median
- A physics or domain formula

Time series

- **Persistence**: tomorrow equals today
- Seasonal naive: same hour last week

Classification

- Majority class (`DummyClassifier`)
- A simple logistic regression

A model that cannot beat persistence on the same rows and metric has not earned its complexity.

PRACTICE

5 min · Pairs

Frame five energy problems

Problem	Task type	One row is	Baseline
Oil rate next month	?	?	?
ESP trip within 7 days	?	?	?
Facies from logs	?	?	?
Day-ahead hourly load	?	?	?
Corrosion rate by segment	?	?	?

1. Fill the table, then name one input for each problem that would leak.

PART 2 OF 6

02

Data, EDA and preprocessing

Look at the data before a model does.

- Data shapes and a well-log display
- Distributions, correlation and missingness
- Quality traps: null codes, unit mixes, depth shifts
- Scaling, encoding and imputation

Energy data come in four shapes

Tabular

- One row per well, asset or event
- Completion tables, work orders

Depth-indexed logs

- A row often every 0.15 m
- Adjacent rows are near-duplicates

Time series

- One row per timestamp per asset
- SCADA, ESP and meter streams

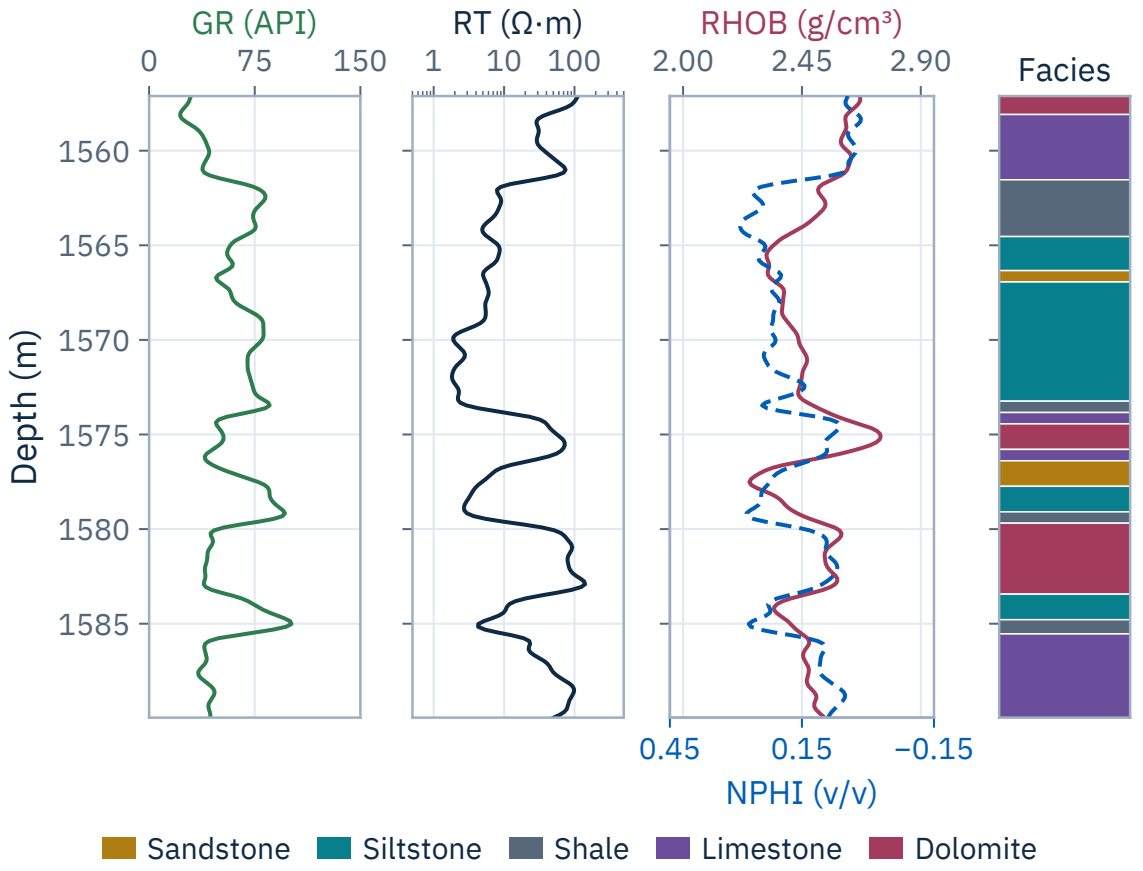
Images and grids

- Pixels with spatial structure
- Seismic sections, core photos (NN Day 4)

◆ The shape decides the split: group rows by well or asset, and respect time order.

A well log is a depth-indexed table

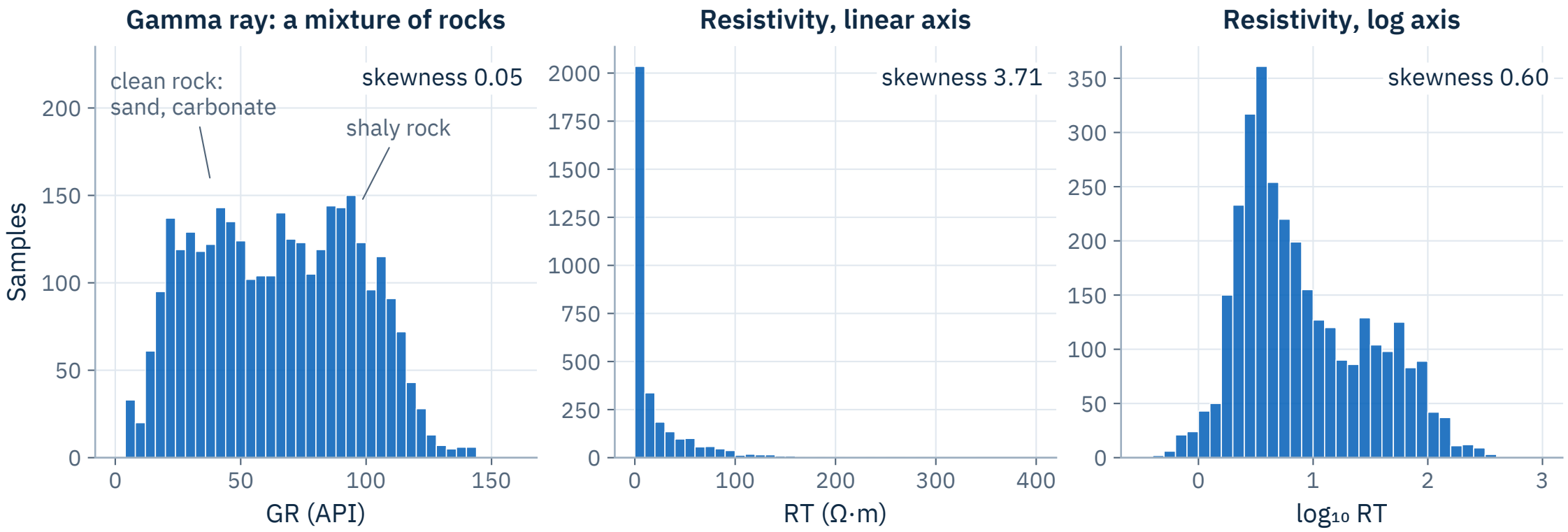
- Each depth sample is one row; each curve is a feature
- Facies from core or interpretation is the target
- Resistivity spans decades, so plot and model $\log_{10}$ RT
- Density and neutron both respond to porosity



Synthetic well W2 (seed 7): 33 m at 0.15 m sampling, generated from facies-dependent distributions. Not a real well.

Distributions reveal mixtures and skew

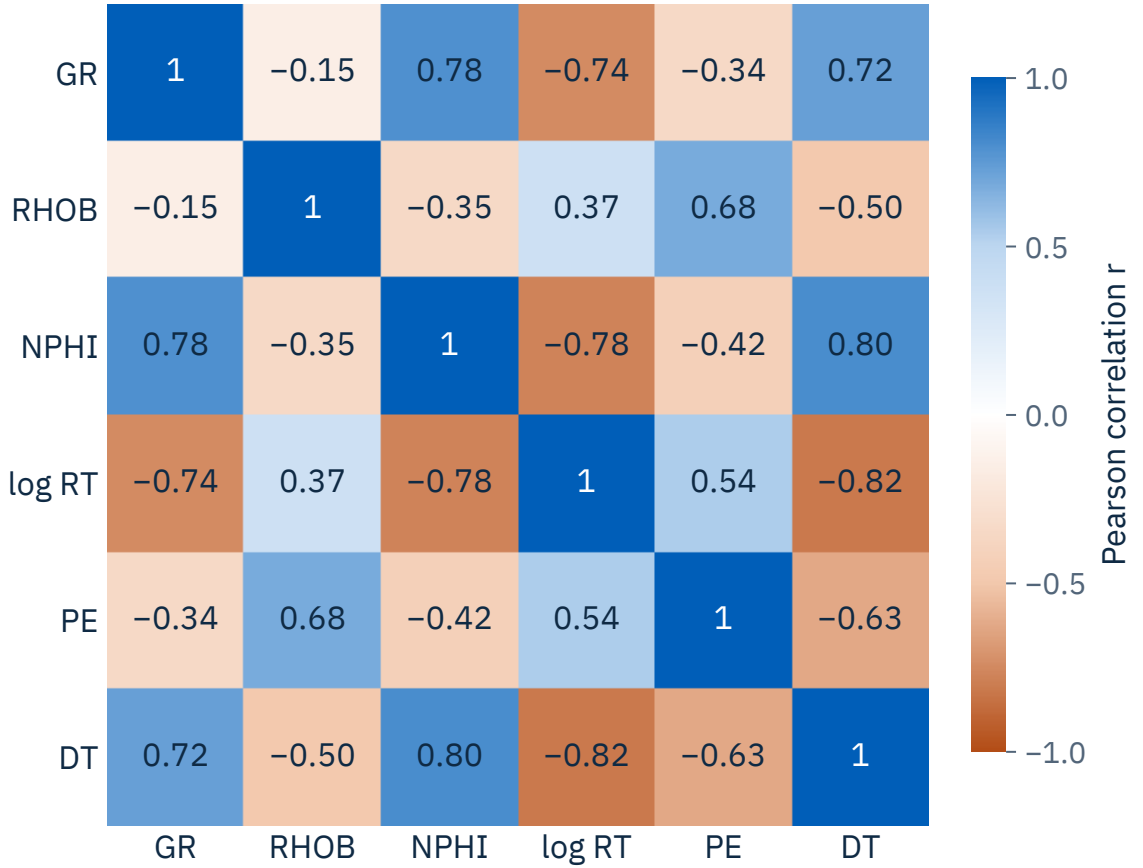
Plot every feature before fitting: mixtures hint at classes, skew hints at a transform.



Computed on the synthetic 8-well dataset (3,200 samples, seed 7); skewness from scipy.stats.skew.

Correlation shows redundancy, not causation

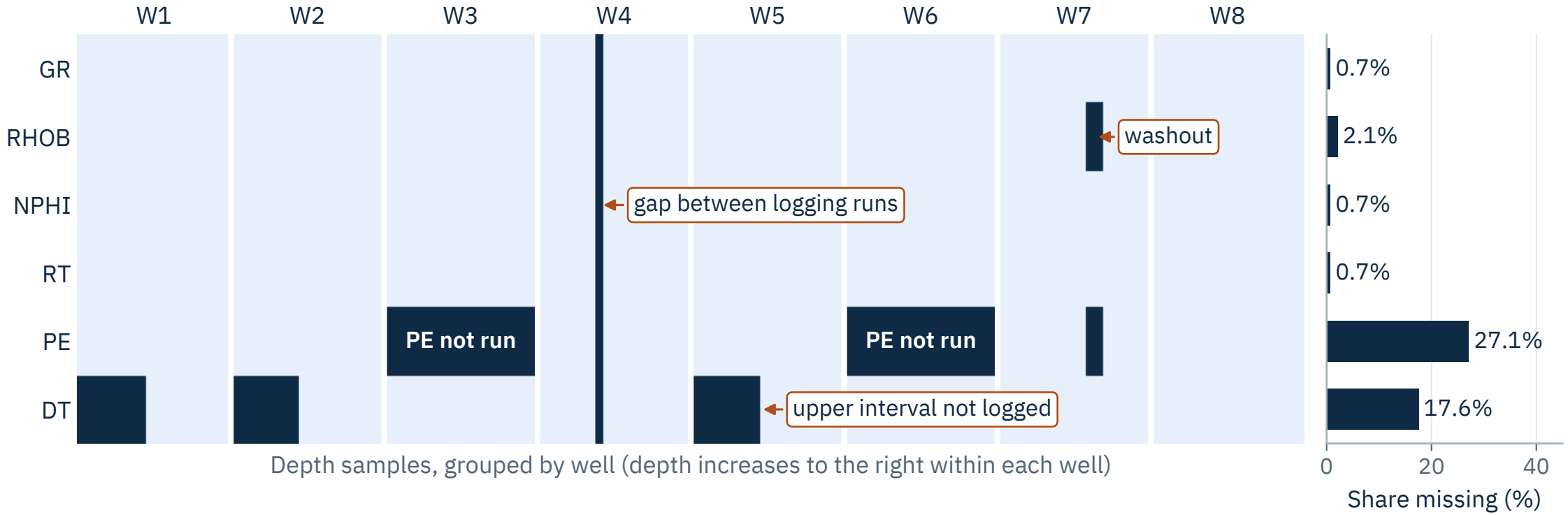
- GR, NPHI and DT rise together (r = 0.72–0.80)
- log RT falls as NPHI and DT rise
- Correlated inputs make linear coefficients unstable
- r near 0 does not rule out a curved relationship



Pearson r over all 3,200 synthetic samples (seed 7). Much of it comes from mixing facies.

Missingness has structure worth recording

Gaps follow tool strings, logging runs and hole conditions: they are patterned, not random.

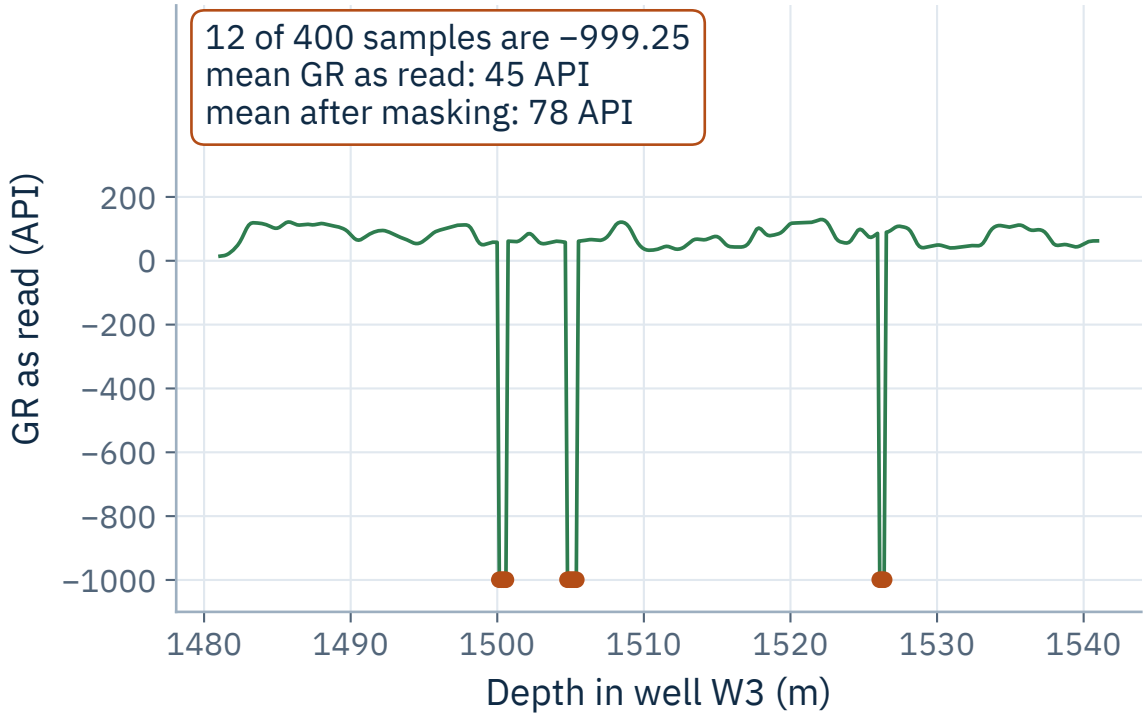


Synthetic dataset (seed 7) with injected, patterned gaps: PE absent in W3 and W6, DT only in the lower interval of W1, W2 and W5, a washout in W7 and a gap between runs in W4.

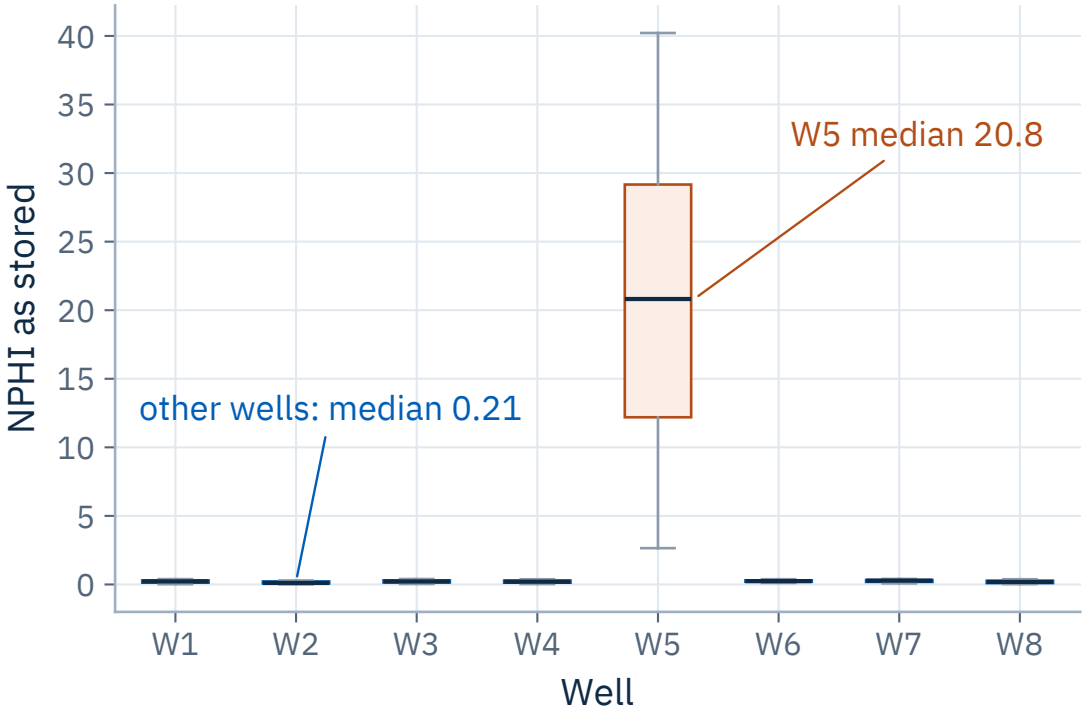
Null codes and unit mixes masquerade as data

Both pass a quick glance at a table; both distort means, scalers and distances.

Null code −999.25 read as a measurement



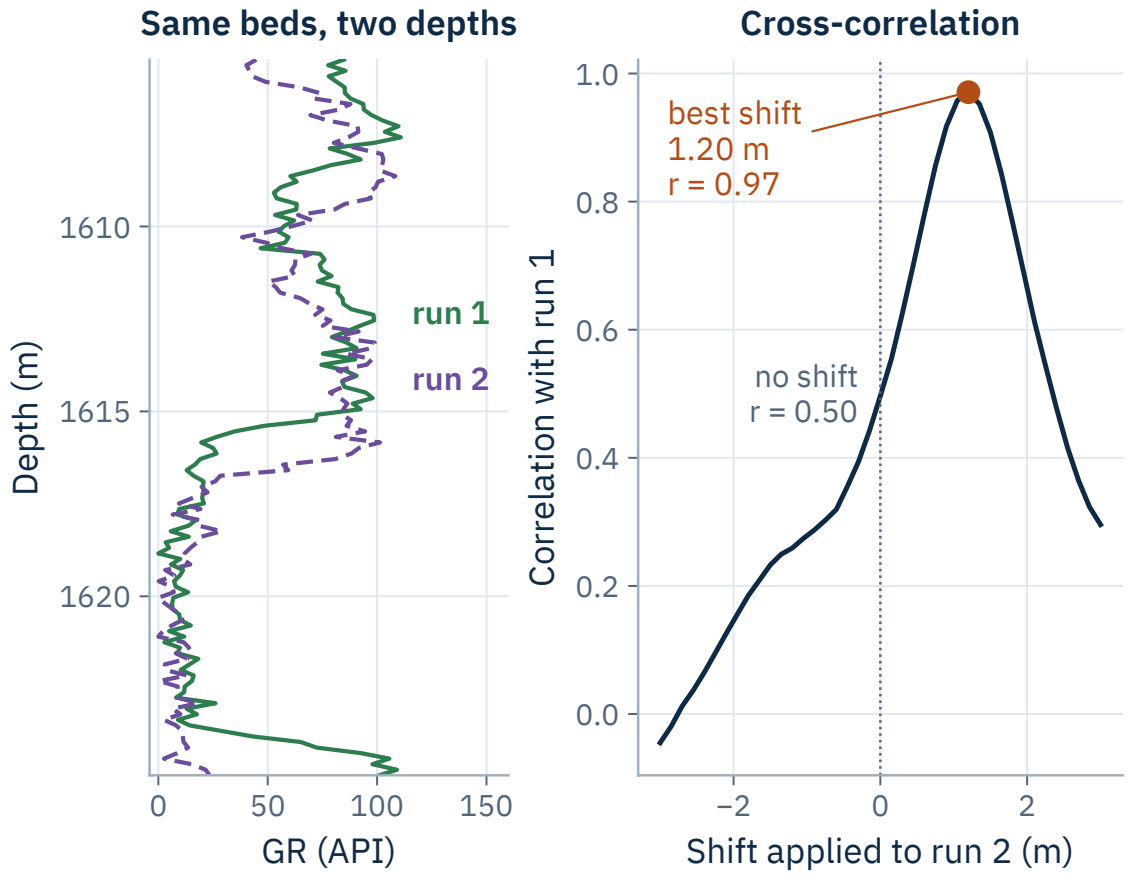
One well stored in percent, the rest in v/v



Synthetic data (seed 7) with injected problems: −999.25 null codes in W3 GR and W5 NPHI stored in percent.

Depth shifts misalign curves from different runs

- Runs can record the same bed at slightly different depths
- **Cross-correlate** a curve logged on both runs, usually GR
- Apply the best shift, then check several beds by eye
- Align before combining curves into features



Synthetic well W4 (seed 7). Run 2 repeats the GR log 1.2 m deeper with independent noise (seed 4); correlation at each trial shift.

PRACTICE

5 min · Pairs

What is wrong with this log summary?

Curve	Rows present of 3,200	Min / median / max
GR (API)	3,178	−999.25 / 65.2 / 142.3
RHOB (g/cm³)	3,133	2.24 / 2.49 / 2.75
NPHI (v/v)	3,178	0.01 / 0.23 / 40.2
RT (Ω·m)	3,178	0.49 / 5.89 / 331.8
PE (b/e)	2,333	1.71 / 3.48 / 6.14
DT (µs/ft)	2,638	45.8 / 90.6 / 128.9

1 Find at least four problems and say what you would check next.

Feature engineering needs physical meaning

✓ Meaningful

- $\log_{10}$ RT instead of raw RT
- Shale volume from GR with per-well clean and shale baselines
- Rolling means within each well, trailing in real time

vs

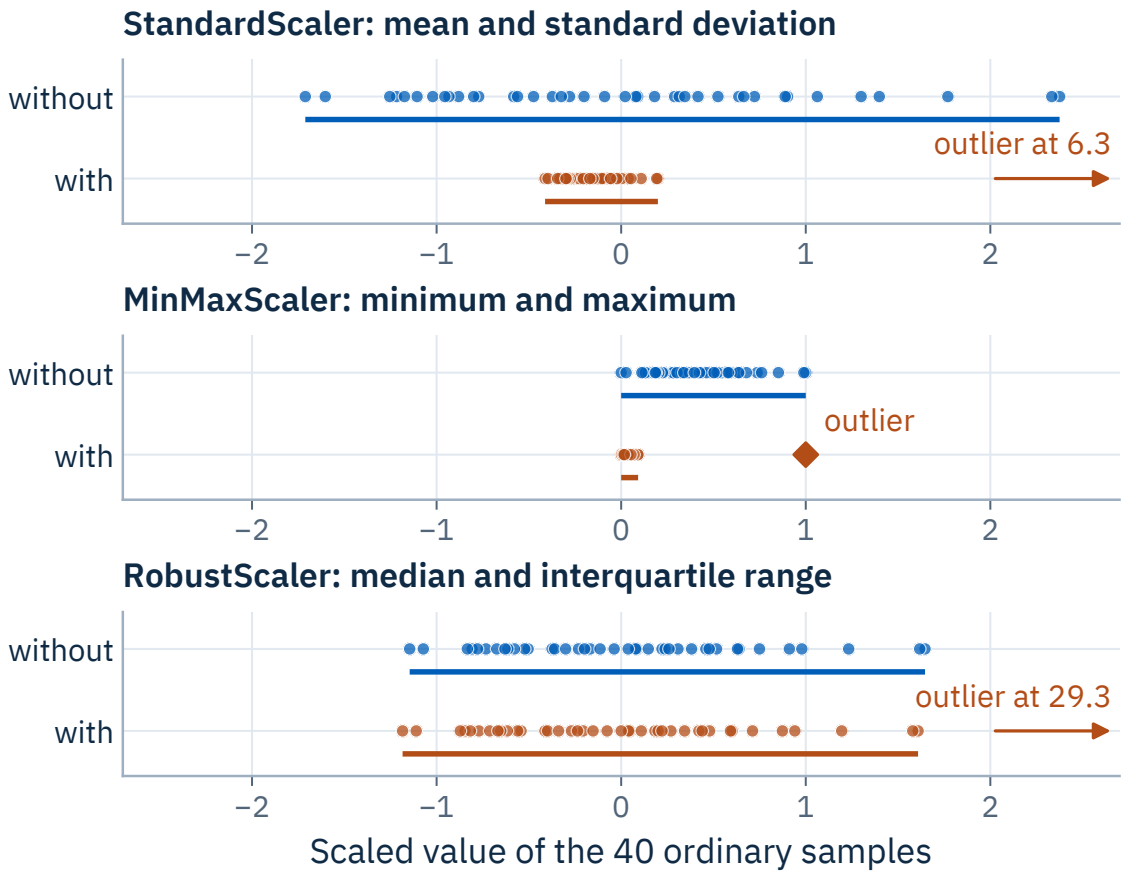
✗ Misleading

- RHOB – NPHI without consistent units or scaling
- Well ID as a predictor for unseen wells
- Centred windows in real-time use: they see the future

◆ Test every engineered feature against a baseline on held-out wells.

Scalers react differently to one outlier

- StandardScaler: mean and SD, both pulled by an outlier
- MinMaxScaler: the range is set by the extremes
- RobustScaler: median and IQR barely move
- Fit any scaler on training rows only



Synthetic feature: 40 samples from N(50, 10), seed 5, plus one outlier at 400. Bars show the range of the 40 ordinary samples.

Encoding and imputation turn columns into numbers

Both steps learn from data, so both belong inside the cross-validated pipeline.

One-hot encoding: one column per category

Formation
A1 SH
A1 LM
B1 SH
A1 LM
B1 SH

→

A1 LM	A1 SH	B1 SH
0	1	0
1	0	0
0	0	1
1	0	0
0	0	1

Integer codes 0, 1, 2 would invent an order that linear models and KNN take literally

Median imputation plus a missing indicator

PE
3.1
NaN
2.8
NaN
4.0

→

PE	PE_missing
3.1	0
3.1	1
2.8	0
3.1	1
4.0	0

median of the training values (2.8, 3.1, 4.0) = 3.1

Both steps are fitted: categories and medians come from training rows only

Schematic worked example; the median 3.1 comes from the three observed training values (2.8, 3.1, 4.0).

PART 3 OF 6

03

Splits, cross-validation and leakage

The split is the experiment.

- Three distinct uses of data
- Cross-validation schemes and what each one estimates
- Parameters versus hyperparameters
- Three forms of leakage, and an exercise

Training, validation and test data do different jobs

Training

- Fits imputation, scaling and model parameters
- Never sees evaluation rows

Validation / CV

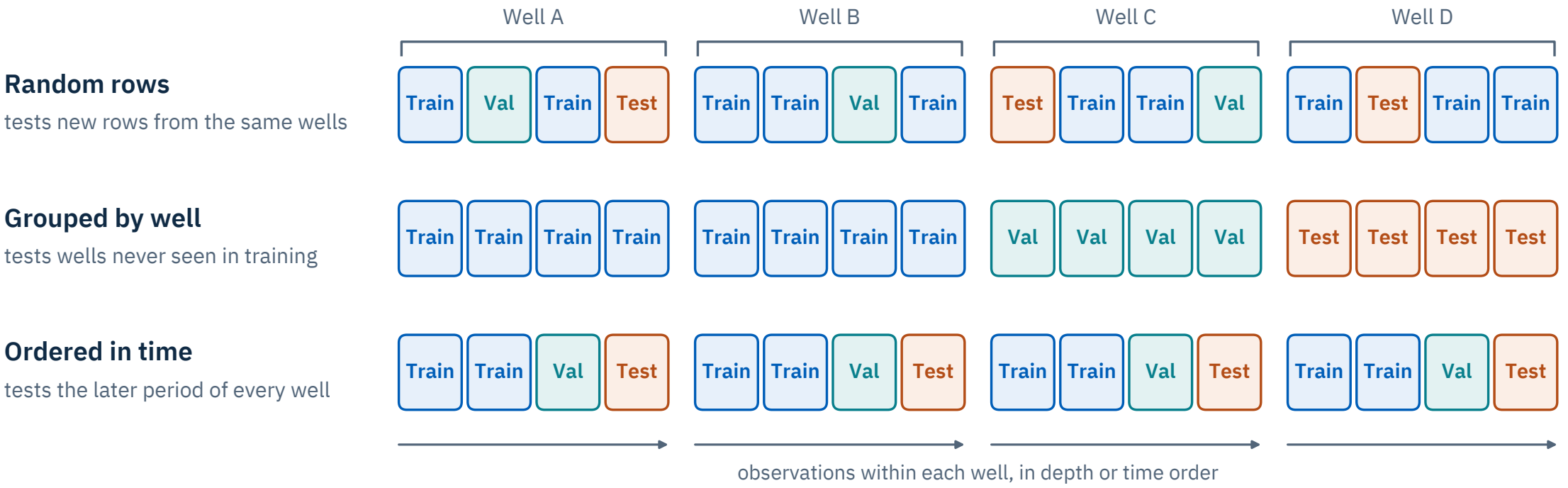
- Selects models, thresholds and settings
- Turns optimistic once you choose on it

Test

- Estimates the locked workflow, once
- Any further tuning turns it into validation

◆ Size the partitions by independent groups and event counts, not by a fixed 60/20/20.

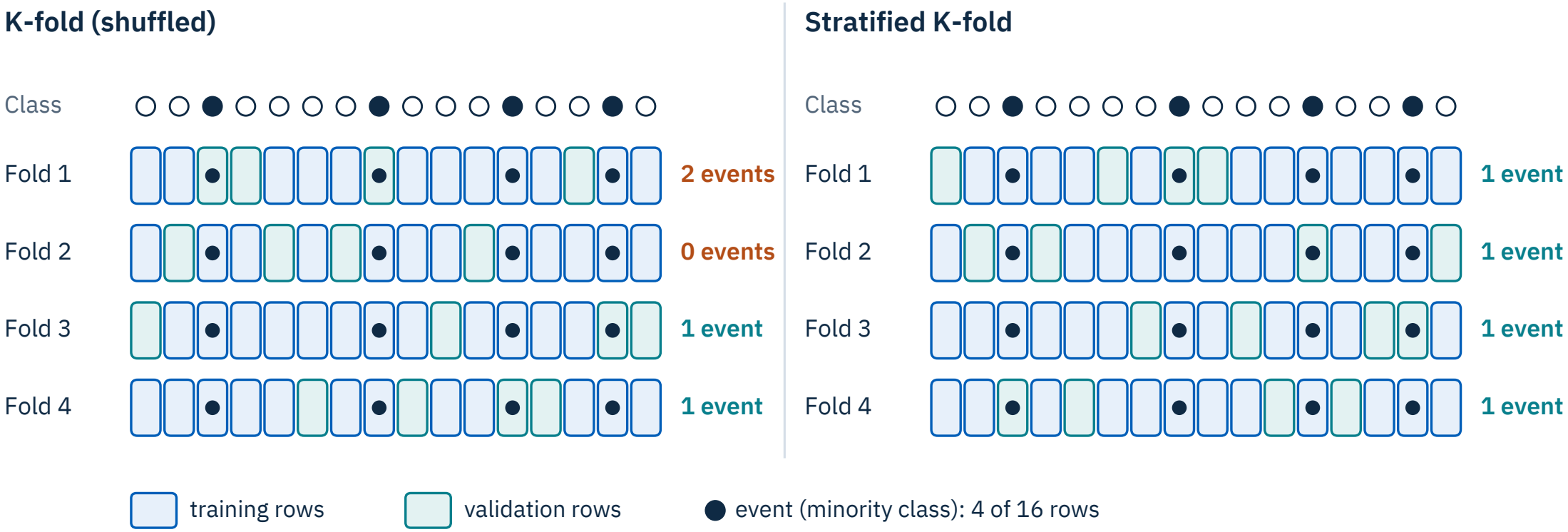
The split follows the deployment question



Schematic split assignments. Each row answers a different generalization question, not a ranking of methods.

K-fold and stratified folds rotate the validation rows

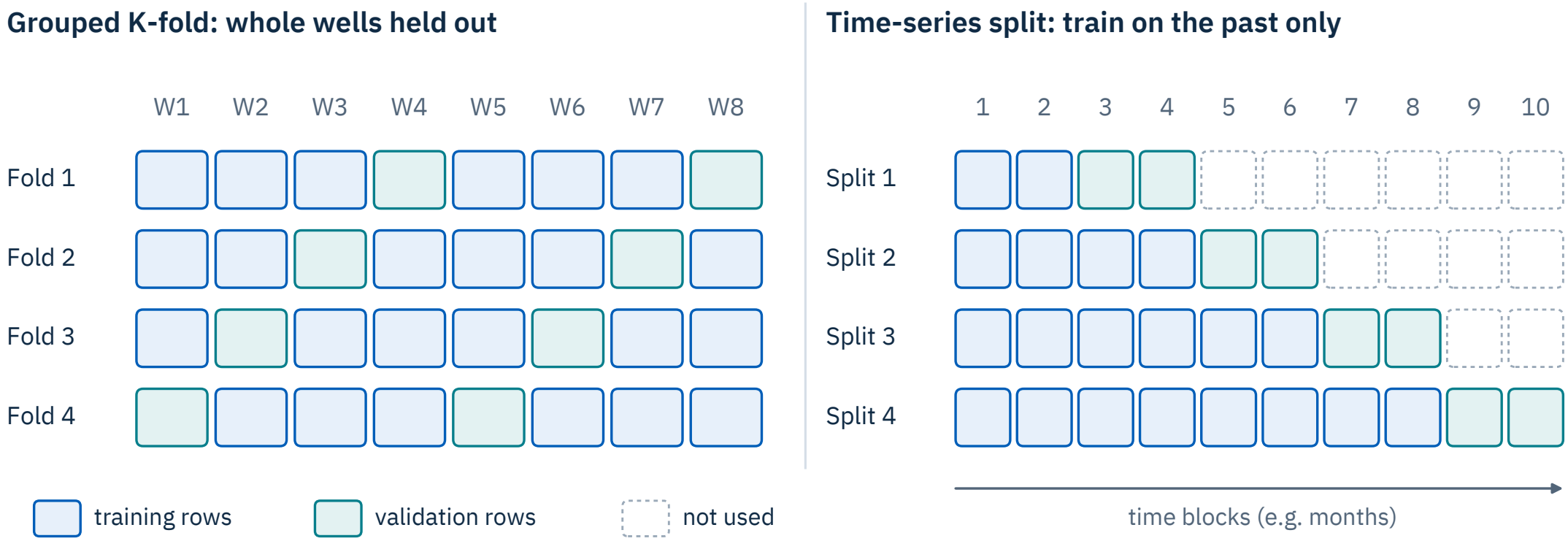
Every row validates exactly once; **stratification** also keeps the class mix in each fold.



Fold assignments computed with scikit-learn KFold and StratifiedKFold (4 folds, shuffle, random_state=1) for 16 rows with 4 events.

Grouped and time-ordered folds mirror deployment

Hold out whole wells for new-well questions; train on the past only for forecasts.



Computed with scikit-learn GroupKFold(4) for 8 equal-size wells and TimeSeriesSplit(4) for 10 time blocks.

Each scheme estimates a different future

Scheme	Estimates performance on	Watch out for
K-fold	New rows like the training rows	Correlated rows inflate scores
Stratified K-fold	The same, with a stable class mix	Still row-level
Grouped K-fold	Wells or assets never seen	Few groups give noisy estimates
Time-series split	The period after training	Needs a gap for the forecast horizon

New wells and future periods at once: group by well and keep time order.

◆ Start from the prediction contract: the deployment question picks the scheme.

Parameters are learned; hyperparameters are chosen

Parameters

- Learned by fit from training rows
- Weights, split thresholds, support vectors
- Also imputer medians and scaler means

vs

Hyperparameters

- Chosen by comparing validation scores
- α, K, C and γ, max_depth
- Also the preprocessing choices themselves

◆ Whatever the validation folds chose, only the test set can judge, and only once.

Leakage takes three common forms

Target leakage

- An input encodes the answer
- A workover flag written after the failure

Train–test contamination

- Evaluation rows shape training
- Scaling all wells; neighbours on both sides

Temporal leakage

- Information from after prediction time
- Centred windows; later-revised allocations

◆ Ask of every input: would this value exist, unchanged, at prediction time?

PRACTICE

3 min · Pairs

Leakage can enter before the model

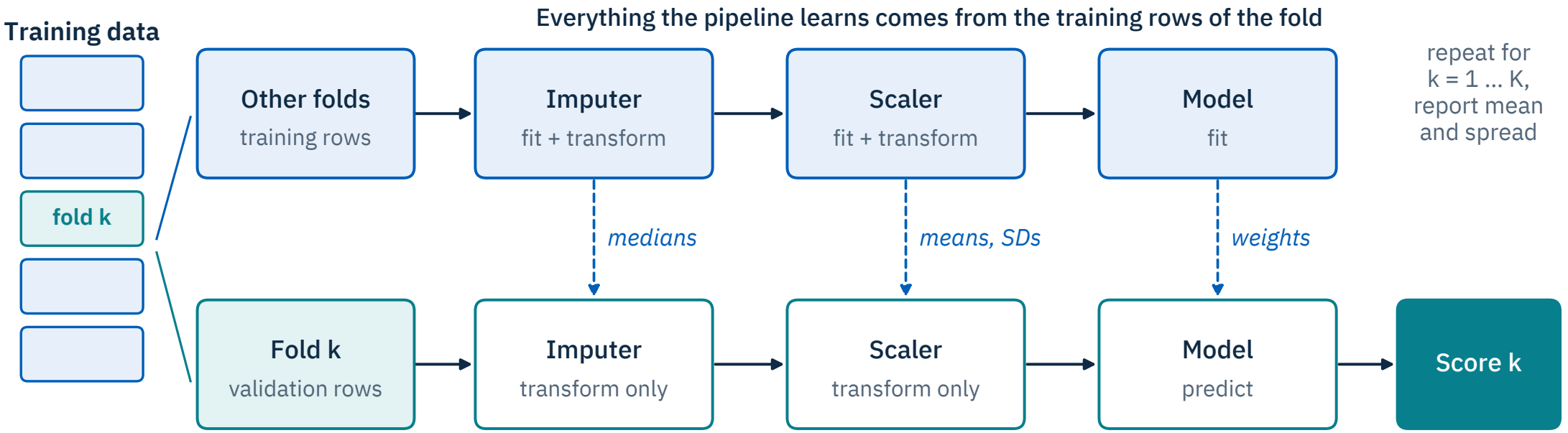
You scale all wells, select features using all labels, then split by well.

1. Which steps have already used the future validation wells?
2. Where should each step run during cross-validation?

DISCUSSION

Fit every step inside the training fold

Answer: refit the imputer, scaler and model on the training rows of each fold; validation rows are only transformed.



Schematic of one cross-validation fold. Module 2 measures how much the wrong order inflates scores.

PART 4 OF 6

04

Linear models and regularization

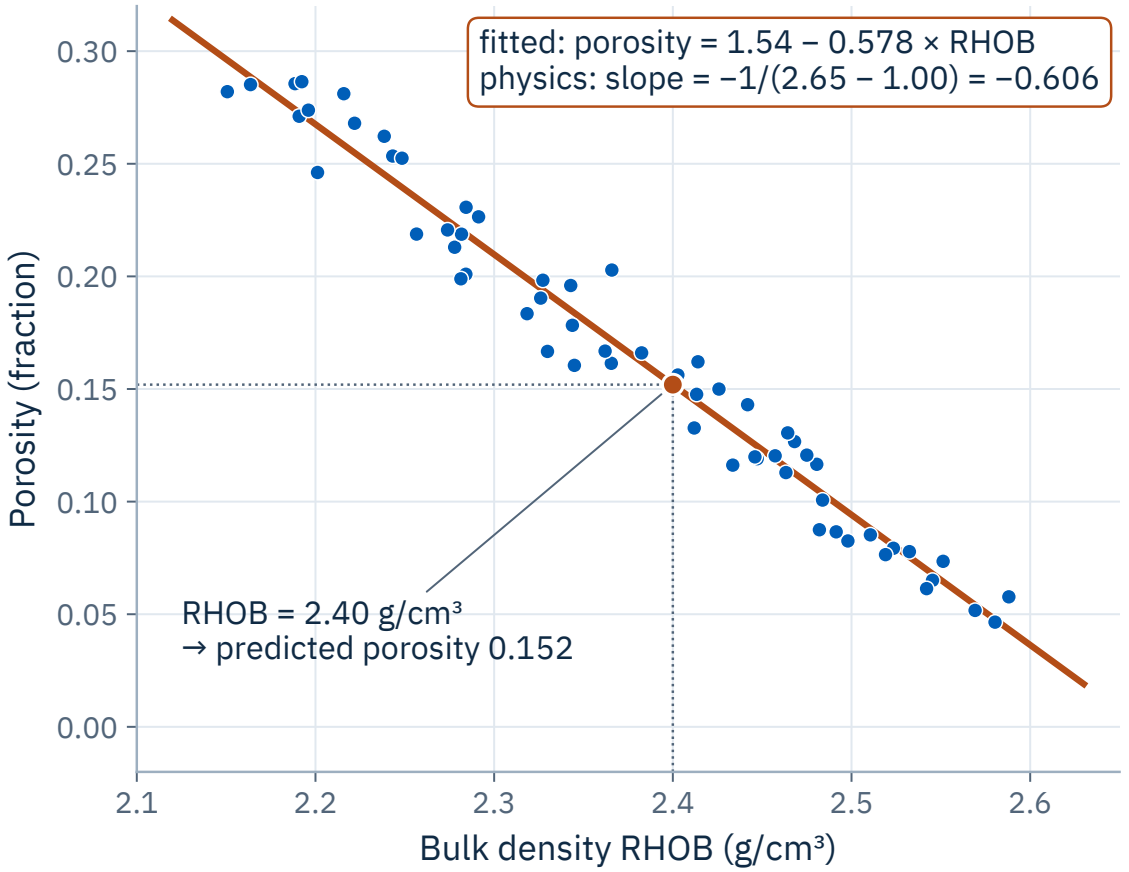
A fitted curve is a hypothesis about unseen observations.

- Linear models, least squares and gradient descent
- Basis expansion, capacity, bias and variance
- Ridge, lasso and robust losses
- Regression metrics

Linear models are linear in their weights

$$\hat{y} = b + \sum_{j=1}^{d} w_j x_j$$

- Each **weight** is the change in prediction per unit of its input, others fixed
- Density–porosity is linear by physics: slope $-1/(\rho_{ma} - \rho_f)$
- Weights depend on units and on correlated inputs



Synthetic data (seed 3) from the density–porosity relation, matrix 2.65 and fluid 1.00 g/cm³.

Least squares chooses the weights

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Least squares picks the weights that minimize L, so each residual counts by its square:

1

cost of a residual of 1

4

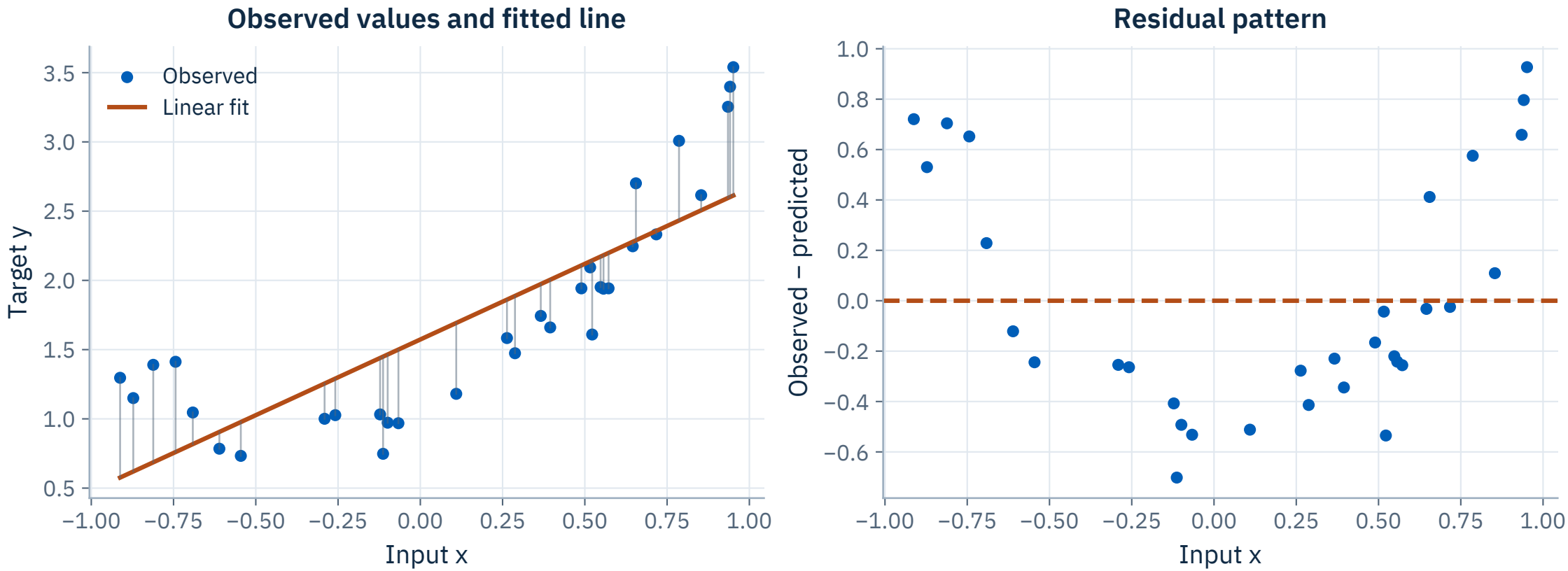
cost of a residual of 2

16

cost of a residual of 4

- Solvers use QR or SVD factorizations, not an explicit matrix inverse.

A residual shows what the model missed



Computed synthetic example. Residual = observed value minus prediction. A curved residual pattern indicates missing structure.

One gradient step, worked by hand

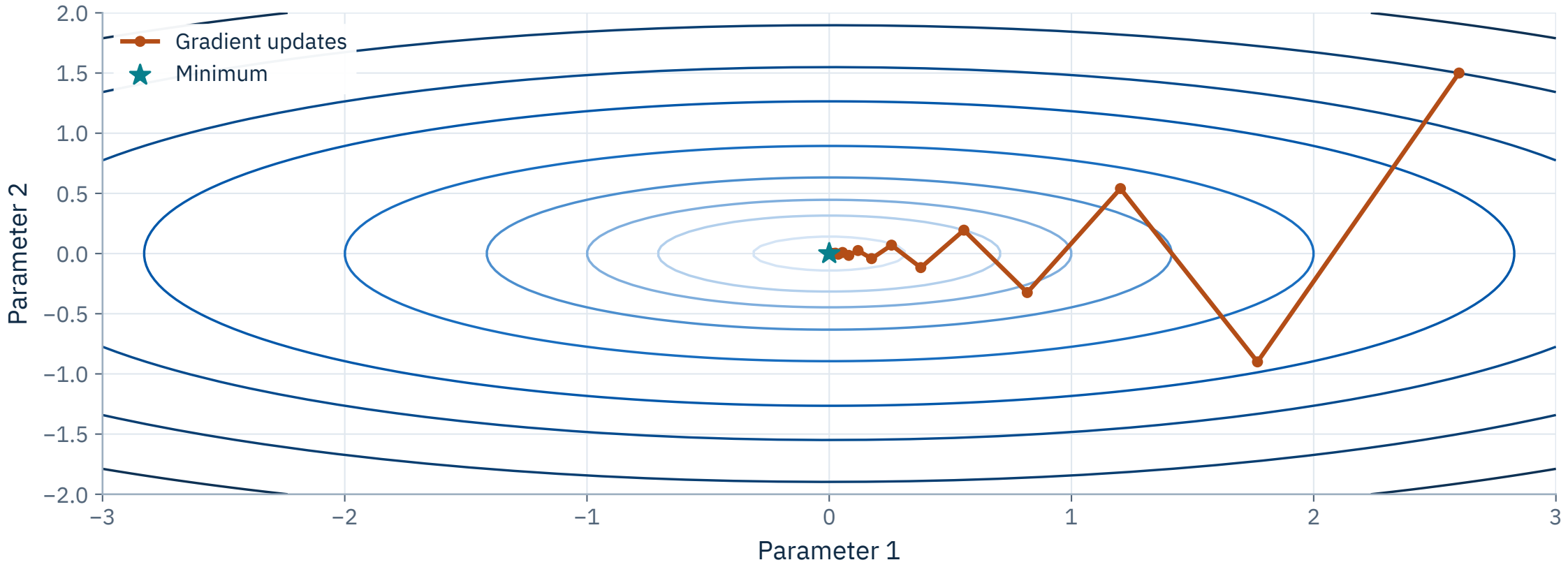
$$w_{new} = w - \alpha \frac{\partial L}{\partial w}$$

One sample x = 2, y = 5; intercept fixed at 0; start at w = 1; learning rate α = 0.1.

1. **Predict** — $\hat{y} = wx = 2$, so the squared error is $(5 - 2)^2 = 9$
2. **Gradient** — $\partial L/\partial w = 2x(wx - y) = 2 \cdot 2 \cdot (2 - 5) = -12$
3. **Step** — $w = 1 - 0.1 \times (-12) = 2.2$
4. **Check** — $\hat{y} = 4.4$, and the squared error falls to $(5 - 4.4)^2 = 0.36$

◆ Too large a learning rate overshoots the minimum and can increase the loss.

Gradient descent zigzags when curvatures differ

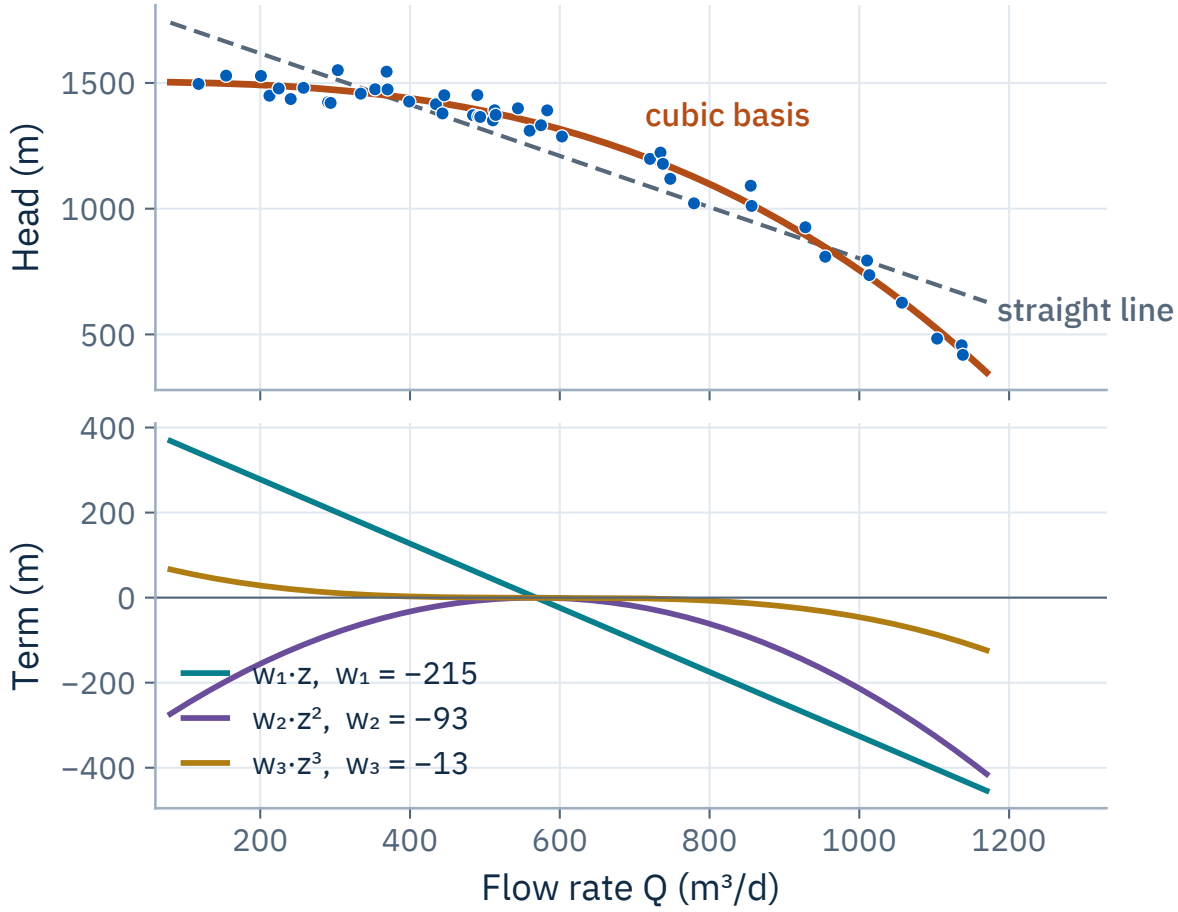


Computed on the exact quadratic loss $L = w_1^2 + 5w_2^2$, learning rate 0.16. The path follows successive updates from one start.

Basis expansion lets a linear model bend

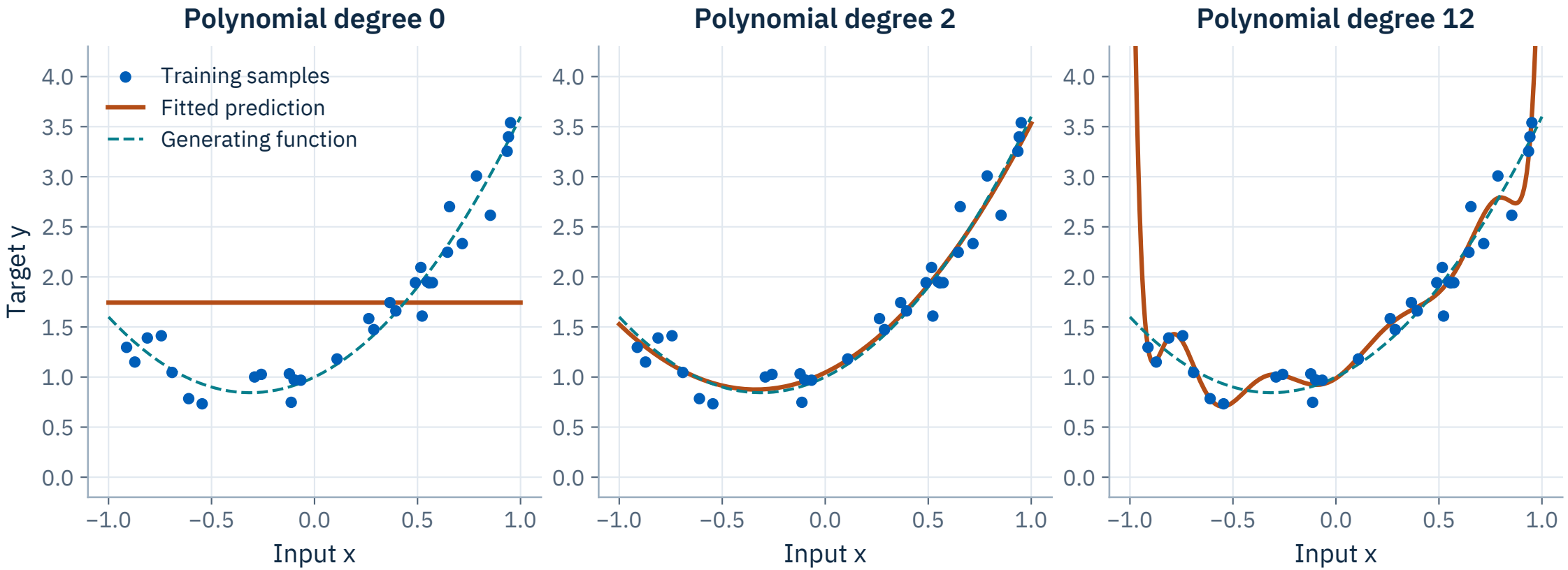
$$\hat{y} = b + w_1 z + w_2 z^2 + w_3 z^3$$

- Still linear in the weights, so least squares still applies
- z is standardized flow: raw powers of Q would be badly conditioned
- The degree is a hyperparameter: choose it by validation



Synthetic head–flow data (seed 8), 45 points; cubic fit on the standardized flow z.

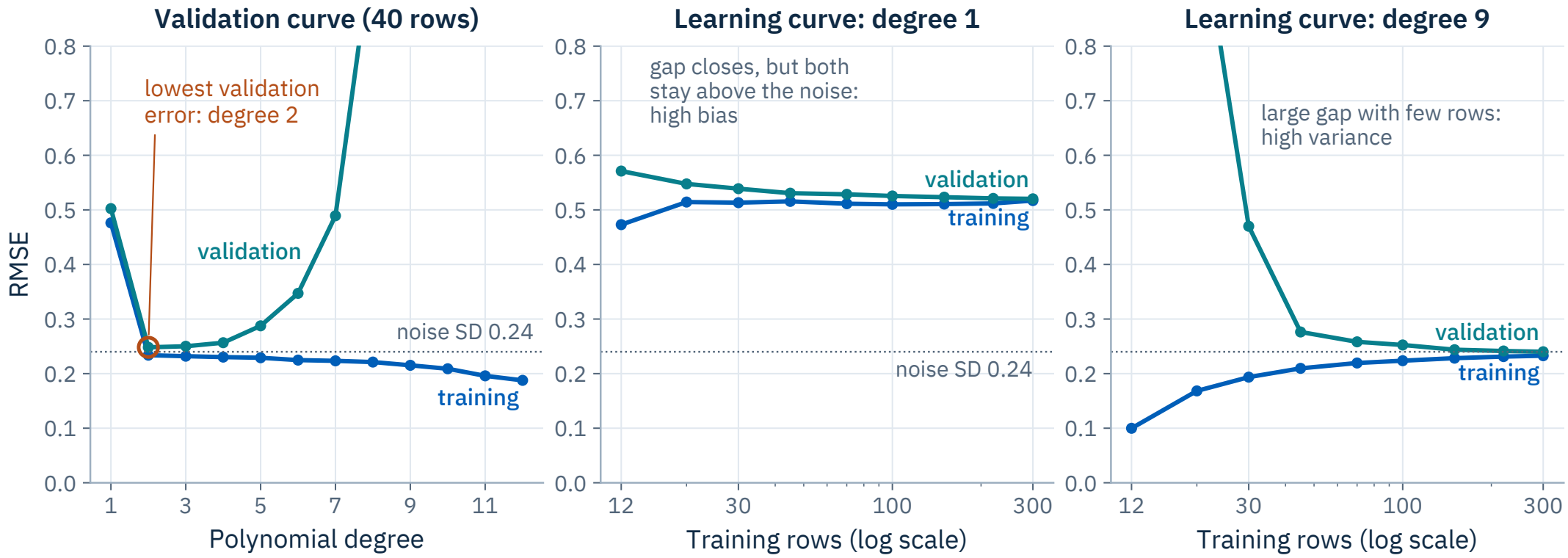
Capacity changes the fitted function



Three models fitted to the same synthetic training observations. The dashed curve is the generating function, not an available training label.

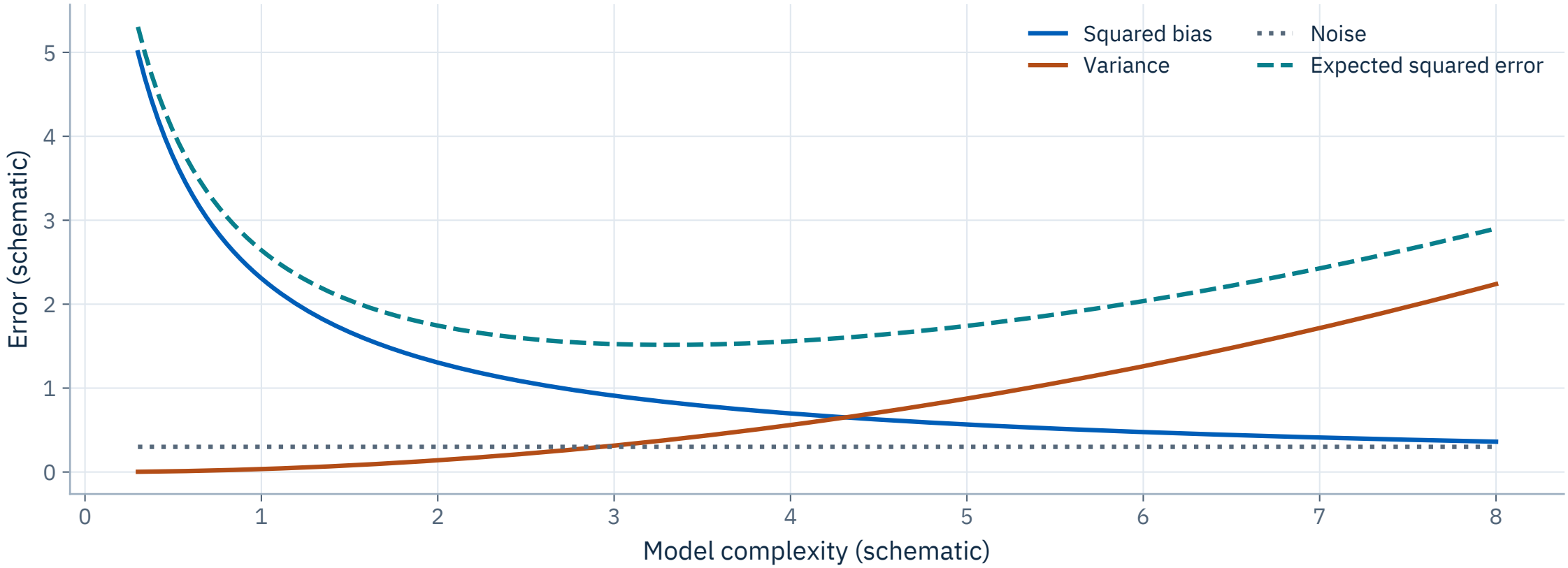
Held-out curves diagnose capacity and data needs

Validation curves choose complexity; learning curves show whether more data would help.



Synthetic $y = 1 + x + 1.6x^2$ + noise (SD 0.24): validation curve on 40 rows (seed 94, 5-fold CV repeated 12 times); learning curves from a 400-row pool (seed 12). Mean RMSE.

Bias and variance explain different failures



Schematic squared-error decomposition. These curves illustrate a concept and are not measured learning curves.

Ridge, lasso and elastic net add a penalty

$$L(w) = \sum_i (y_i - \hat{y}_i)^2 + \alpha\, P(w)$$

Ridge (L2)

- $P(w) = \sum_j w_j^2$
- Shrinks all weights; keeps correlated ones

Lasso (L1)

- $P(w) = \sum_j |w_j|$
- Sets some weights exactly to zero

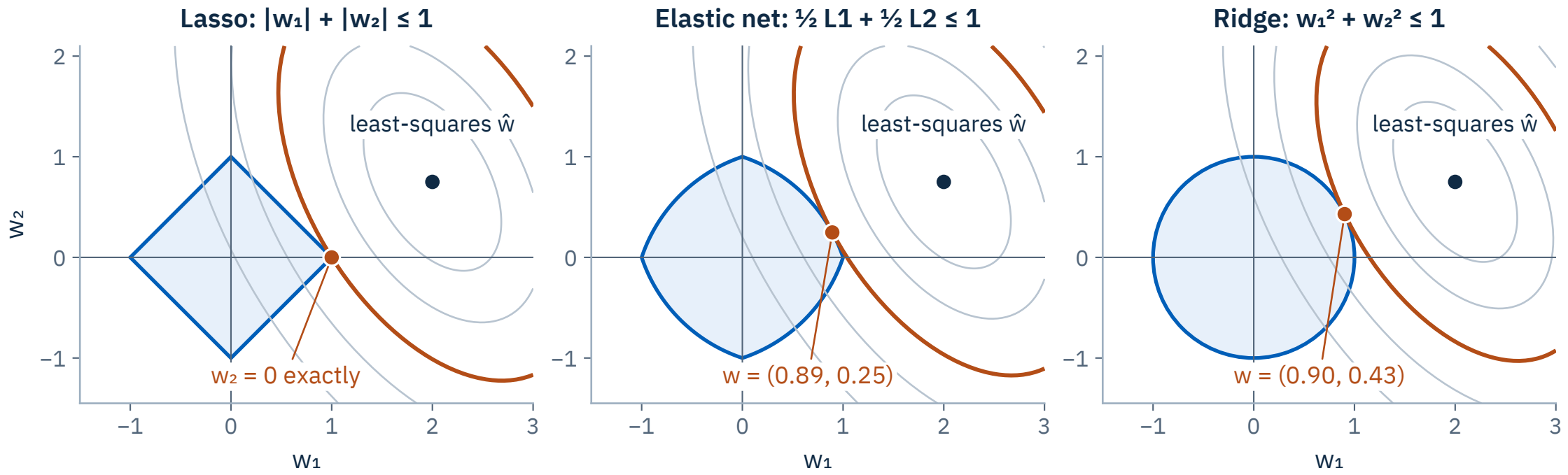
Elastic net

- A weighted mix of both penalties
- Sparse, and steadier with correlated inputs

◆ Standardize predictors first and tune α inside cross-validation; the intercept is not penalized.

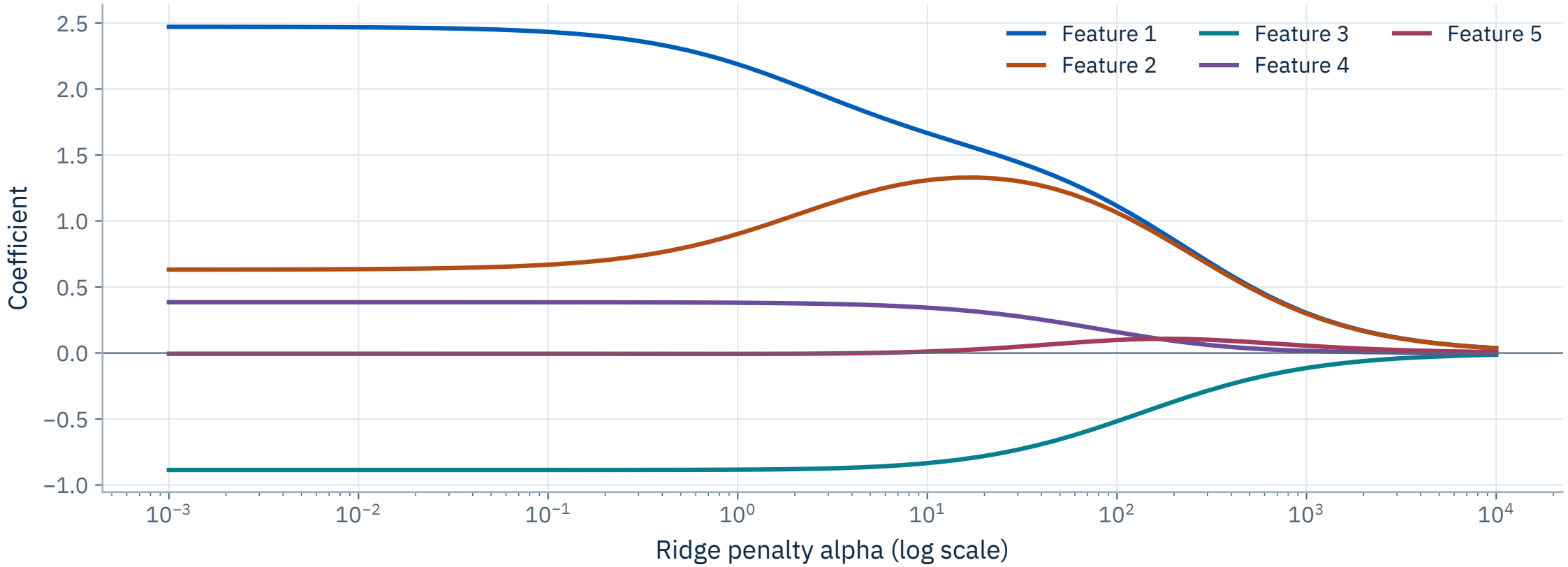
L1 and L2 penalties have different geometry

The fit sits where the smallest loss contour touches the allowed region: corners on the axes give exact zeros.



Exact quadratic loss centred at ŵ = (2.0, 0.75); constrained minima computed numerically. Elastic net uses half L1, half L2.

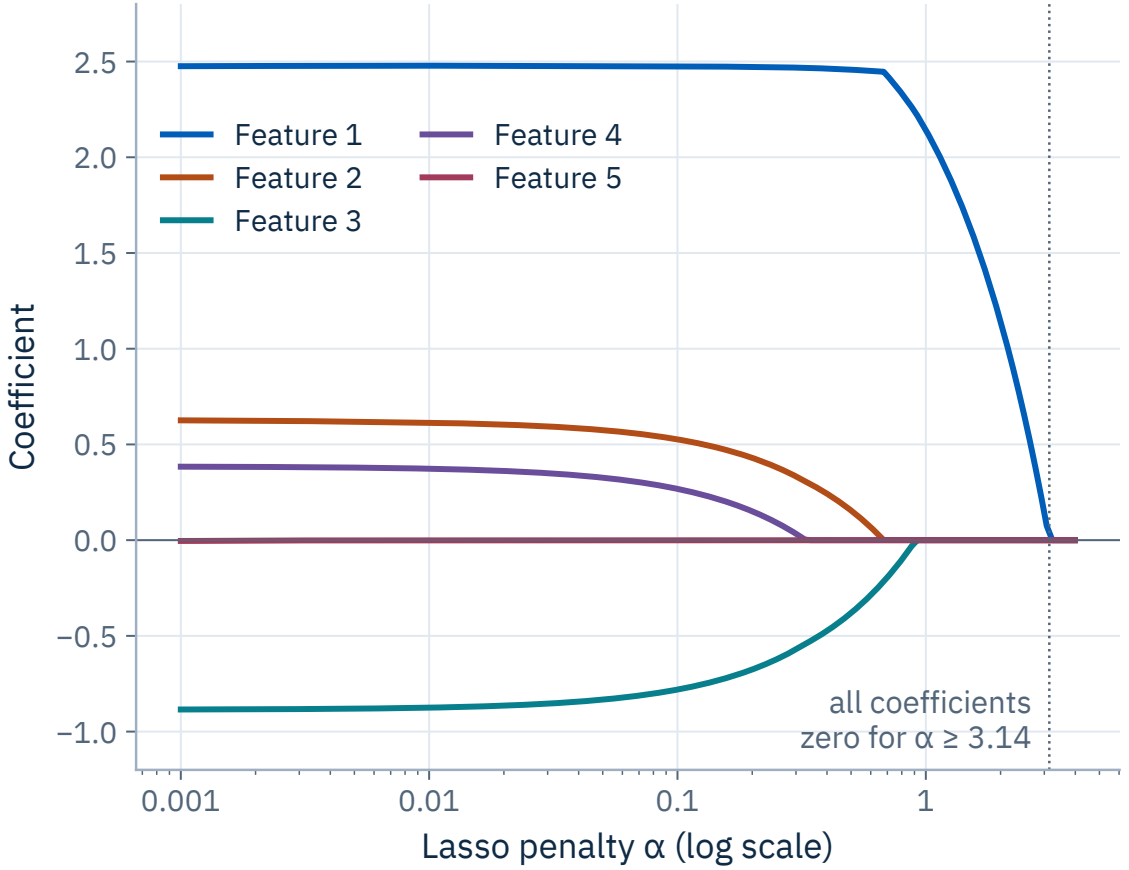
Coefficients shrink as the penalty grows



Ridge fits on one standardized synthetic dataset. The horizontal axis is logarithmic. Lines are coefficients, not importance scores.

Lasso sets coefficients exactly to zero

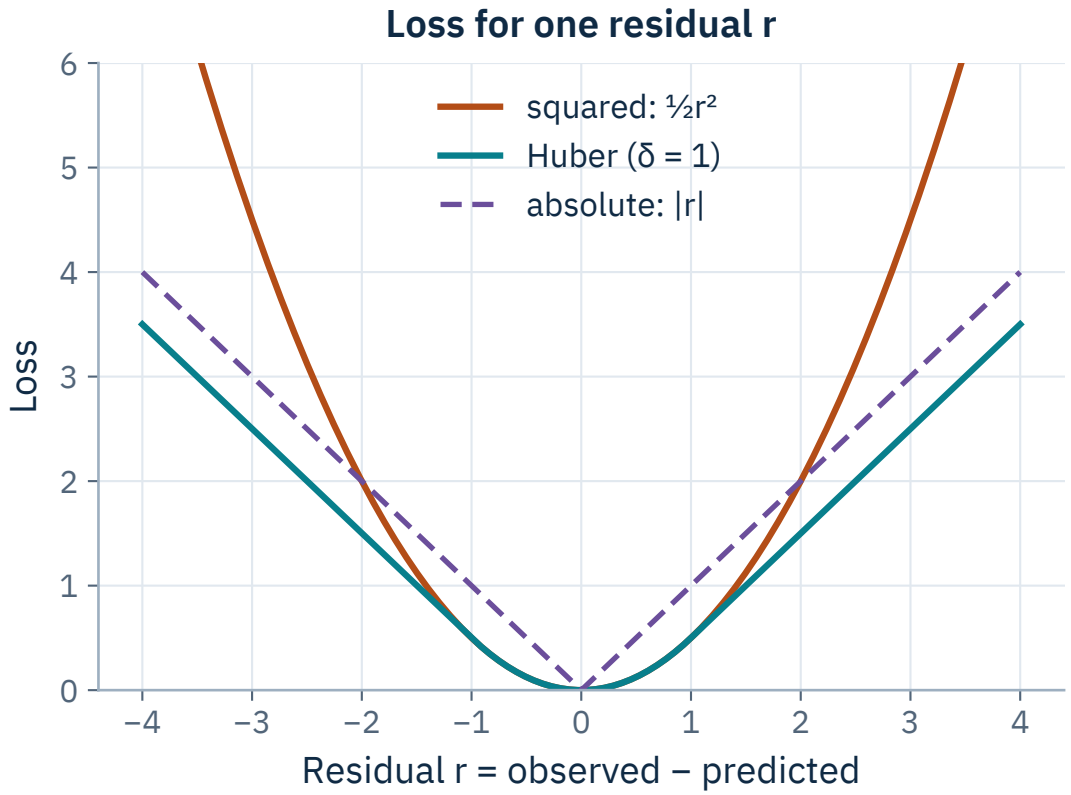
- Same standardized data as the ridge path
- As α decreases, weights leave zero one at a time
- Of the correlated features 1 and 2, lasso drops feature 2 first
- Above α = 3.14 every weight is zero

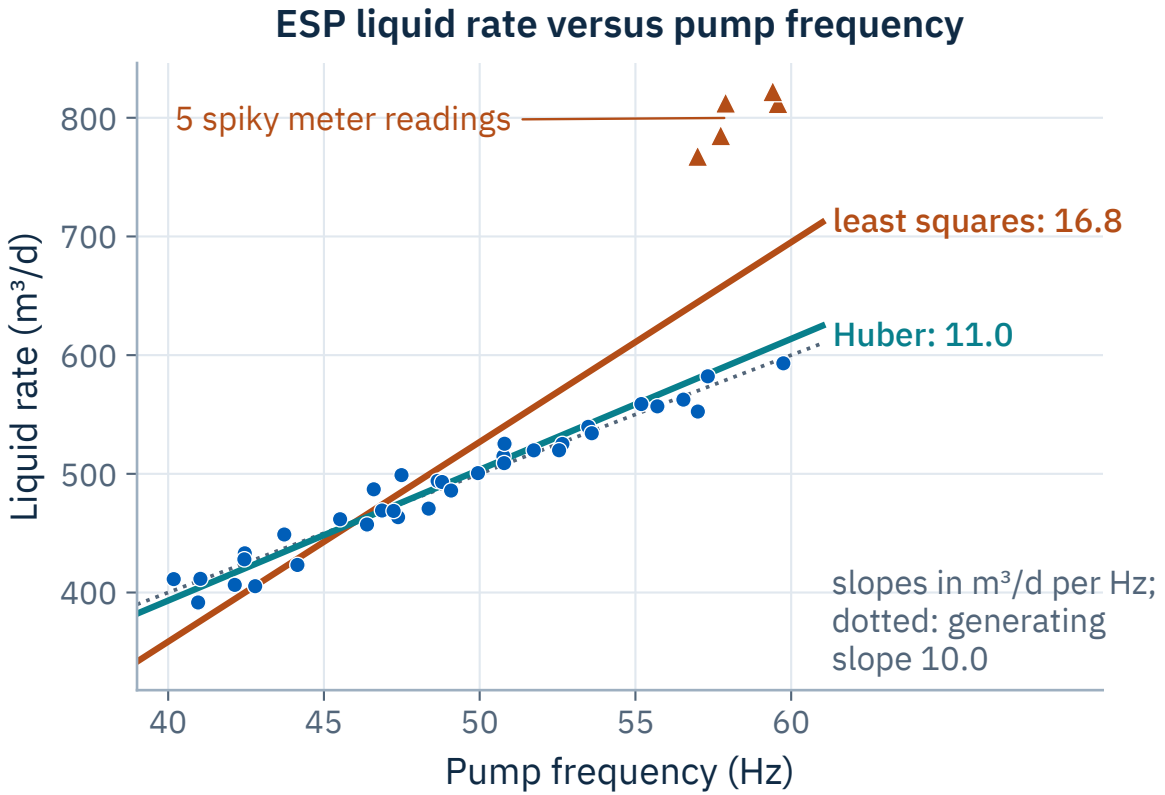


Same synthetic data as the shared ridge path (seed 42); computed with scikit-learn lasso_path.

Squared loss lets a few spikes pull the fit

Huber loss is quadratic for small residuals and linear for large ones, so spikes pull less.





Synthetic ESP data (seed 6): rate = 10 × frequency + noise, with 5 upward spikes. Slopes: least squares 16.8, HuberRegressor 11.0 m³/d per Hz.

Regression metrics answer different questions

$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

MAE

- Average miss, in target units
- Treats all error sizes linearly

RMSE

- In target units; large misses count more
- Sensitive to outliers

R²

- Improvement over the evaluation mean
- Can be negative; depends on the spread

◆ Report errors by well or operating regime as well as overall.

PRACTICE

4 min · Individual

Compute MAE, RMSE and R^2 by hand

Observed y = 10, 12, 9, 15; predicted $\hat{y}$ = 9, 13, 9, 11.

1. Compute the residuals, the MAE and the RMSE.
2. Compute R^2, given that the mean of y is 11.5.
3. Which metric reacts most to the one large miss?

LABORATORY

40 min

Lab 1: regression and regularization

- §1.1–1.2: split three ways; fit linear and degree-2 models; record R^2 and MAE
- Put a DummyRegressor mean baseline in the first row of your score table
- §1.3: plot training and validation R^2 against degree, raw and standardized
- §2: compare ridge and lasso over α on validation rows; count nonzero terms
- §2.3: lock one model, then score the test rows once

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table, baseline first
- Degree plots: raw powers and standardized columns
- One sentence: which α, and how it was chosen

DISCUSSION

Lab 1 debrief: what the regression lab shows

Reference run of the solution (16,512 training, 2,064 validation, 2,064 test rows; mean baseline R^2 0.00):

0.59

validation R^2, linear on 8 features

0.66

validation R^2, degree 2 (44 terms)

0.11

training R^2, raw MedInc at degree 15

0.63

test R^2 of the locked ridge, used once

- Training R^2 cannot fall as the degree grows unless the solver fails: standardize the polynomial columns.

PART 5 OF 6

05

Classification models

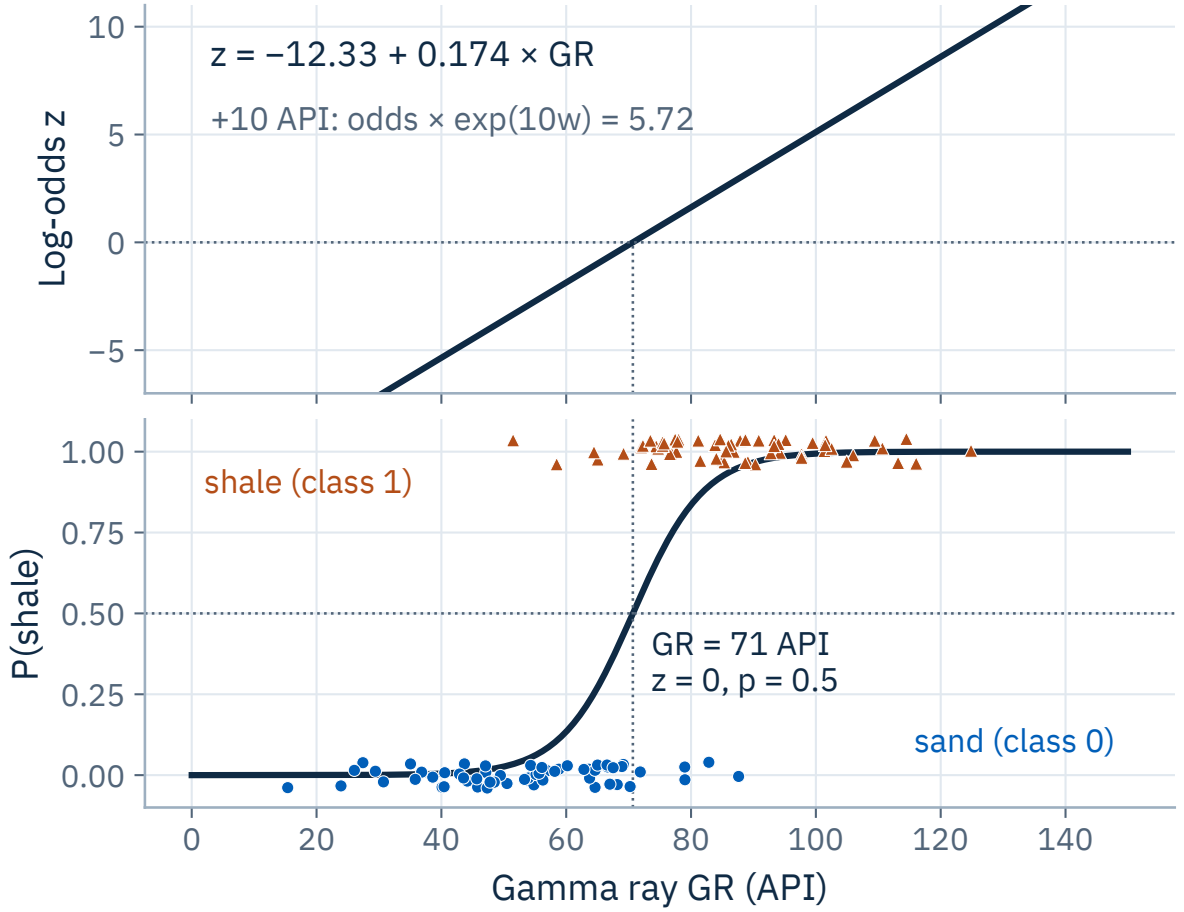
Each model makes a different assumption about the boundary.

- Logistic regression
- Nearest neighbours
- Support vector machines
- Decision trees

Logistic regression maps a score to a probability

$$p = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad z = b + w^{\top} x$$

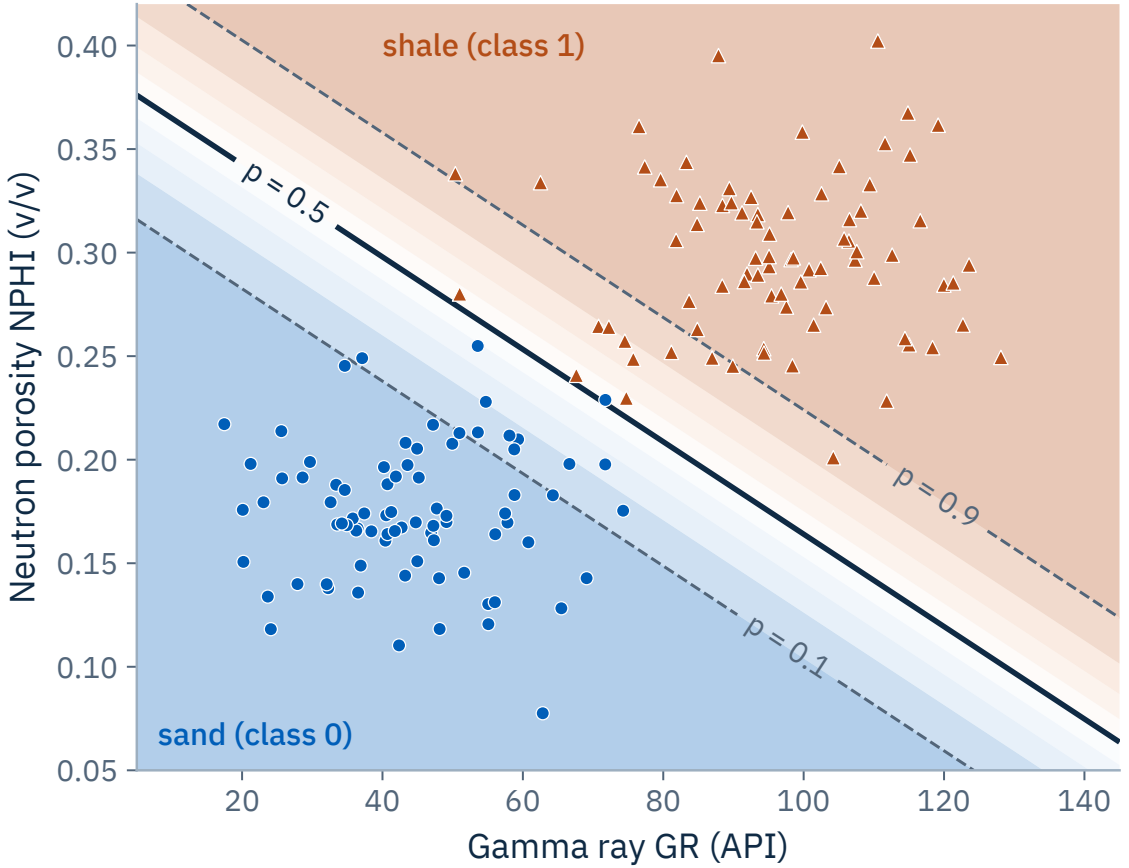
- z is the **log-odds**, ln(p / (1 – p)), linear in the inputs
- Each +1 in an input multiplies the odds by e^{w}
- p = 0.5 exactly where z = 0



Synthetic sand and shale GR (60 each, seed 2); fitted weight 0.174 per API.

The decision boundary is where p equals 0.5

- With two inputs the **decision boundary** z = 0 is a straight line
- Probability contours run parallel to it
- Curved boundaries need basis expansion or another model

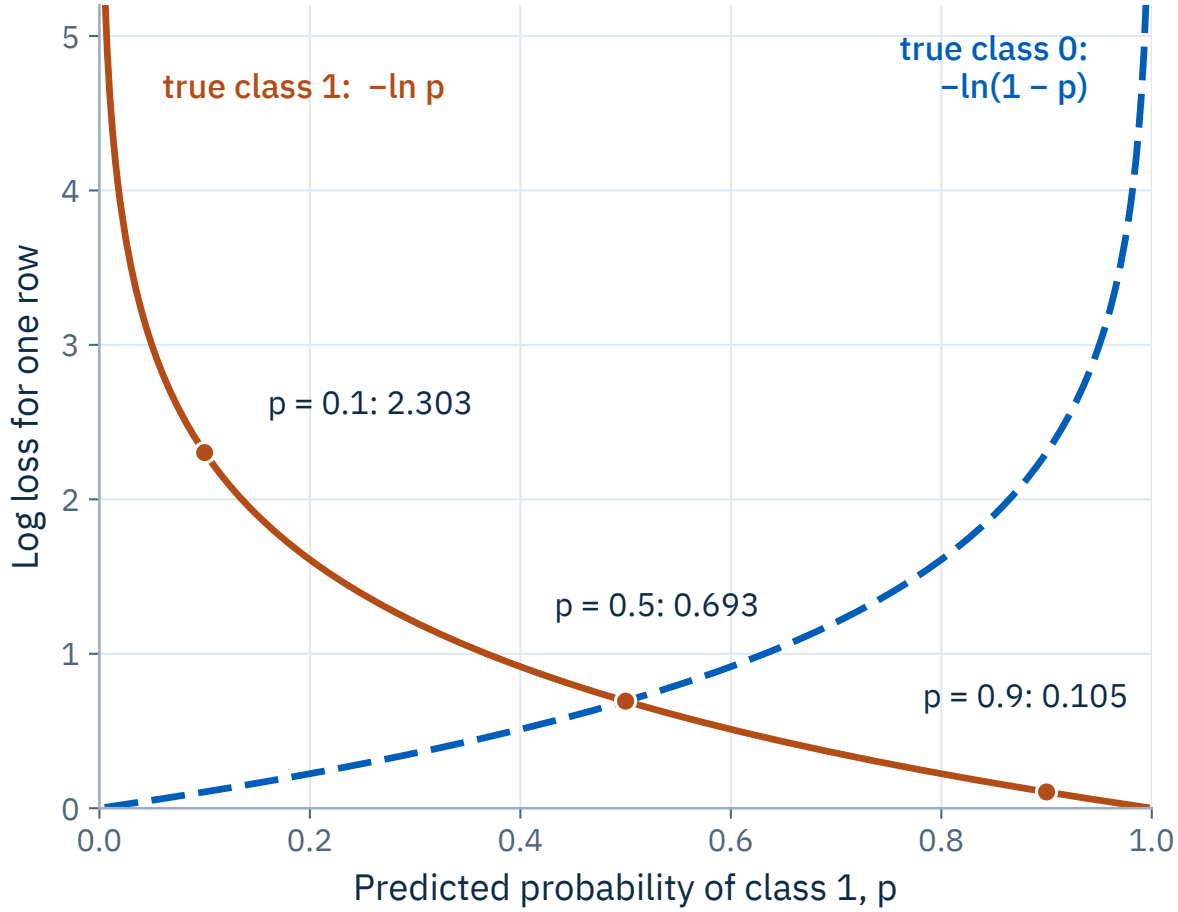


Synthetic sand and shale samples (80 each, seed 4); standardized logistic regression; contours of predicted probability.

Log loss punishes confident mistakes

$$L = -[y \ln p + (1 - y) \ln(1 - p)]$$

- True shale predicted at p = 0.9 costs 0.105
- The same shale at p = 0.1 costs 2.303, about 22 times more
- Fitting minimizes the mean log loss with an iterative solver



Exact functions; marked values −ln 0.9, −ln 0.5 and −ln 0.1.

A worked prediction from two standardized logs

Weights b = −0.4, w_{GR} = 1.5, w_{RHOB} = 0.8; one sample at GR = 1.2 and RHOB = −0.5 (z-scores).

1. **Score** — $z = -0.4 + 1.5 \times 1.2 + 0.8 \times (-0.5) = -0.4 + 1.8 - 0.4 = 1.0$
2. **Probability** — $\sigma(1.0) = 1 / (1 + e^{-1}) \approx 0.73$
3. **Decision** — $0.73 \geq 0.5$, so predict shale
4. **Loss if the sample is shale** — $-\ln 0.73 \approx 0.31$
5. **Loss if the sample is sand** — $-\ln 0.27 \approx 1.31$

◆ GR adds +1.8 and RHOB −0.4 to the score: such comparisons only make sense on standardized inputs.

Scale, fit, then read probabilities with predict_proba

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=1.0, max_iter=1000),
)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_val)[:, 1]
pred = (proba >= 0.5).astype(int)
```

- C is the inverse L2 strength: smaller C means stronger regularization
- More than two classes: a softmax over classes, by default
- Choose thresholds on validation rows, not 0.5 by habit

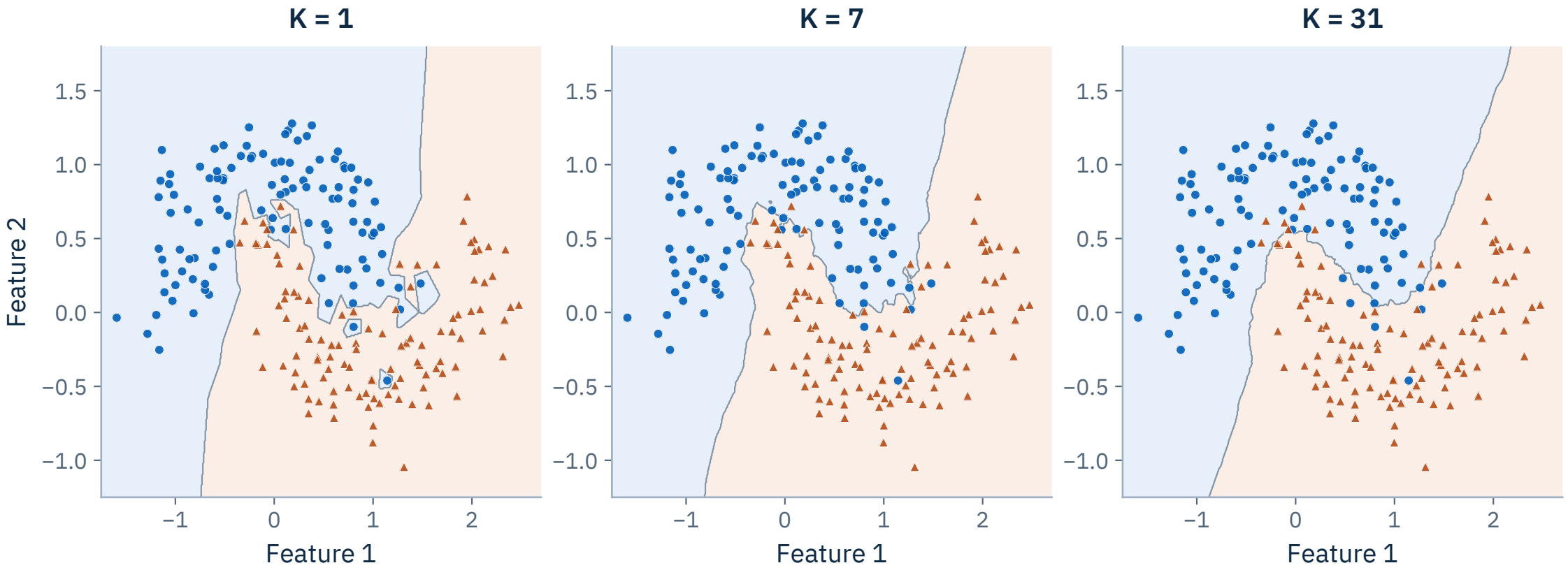
KNN predicts from the nearest labelled examples

Worked vote with K = 5: neighbour labels 1, 0, 1, 1, 0 give class 1, three votes to two.

1. **Measure**
   distance from the query to every training row
2. **Select**
   the K nearest rows
3. **Vote or average**
   majority class, or mean target for regression
4. **Predict**
   break ties by rule or by distance weights

- Standardize with training statistics when units differ; KNN stores rows instead of fitting weights.

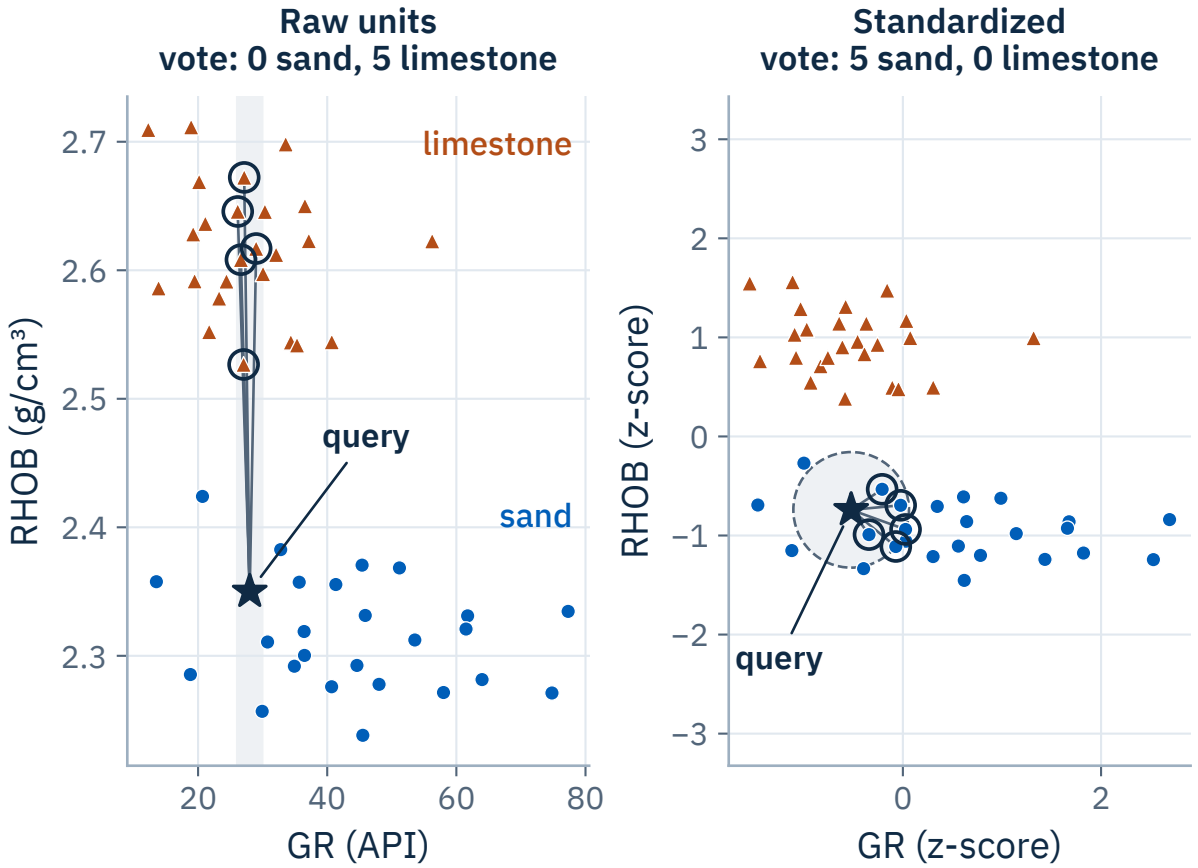
K controls the boundary's detail



Fitted classifiers on the same synthetic data. Backgrounds show predictions; point shape and colour show observed class.

Scaling changes who your neighbours are

- GR spans tens of API units; RHOB spans tenths of a g/cm³
- Raw distance is almost pure GR: all 5 neighbours are limestone
- After standardization RHOB counts: all 5 neighbours are sand



Synthetic sand and limestone (25 each, seed 9); query at GR 28 API, RHOB 2.35 g/cm³ (star); K = 5.

Distance is a modelling choice

Distance between two rows a and b over features j: KNN, RBF kernels and clustering all depend on it.

Euclidean

- $\sqrt{(\sum_j (a_j - b_j)^2)}$
- Large differences dominate; scale matters

Manhattan

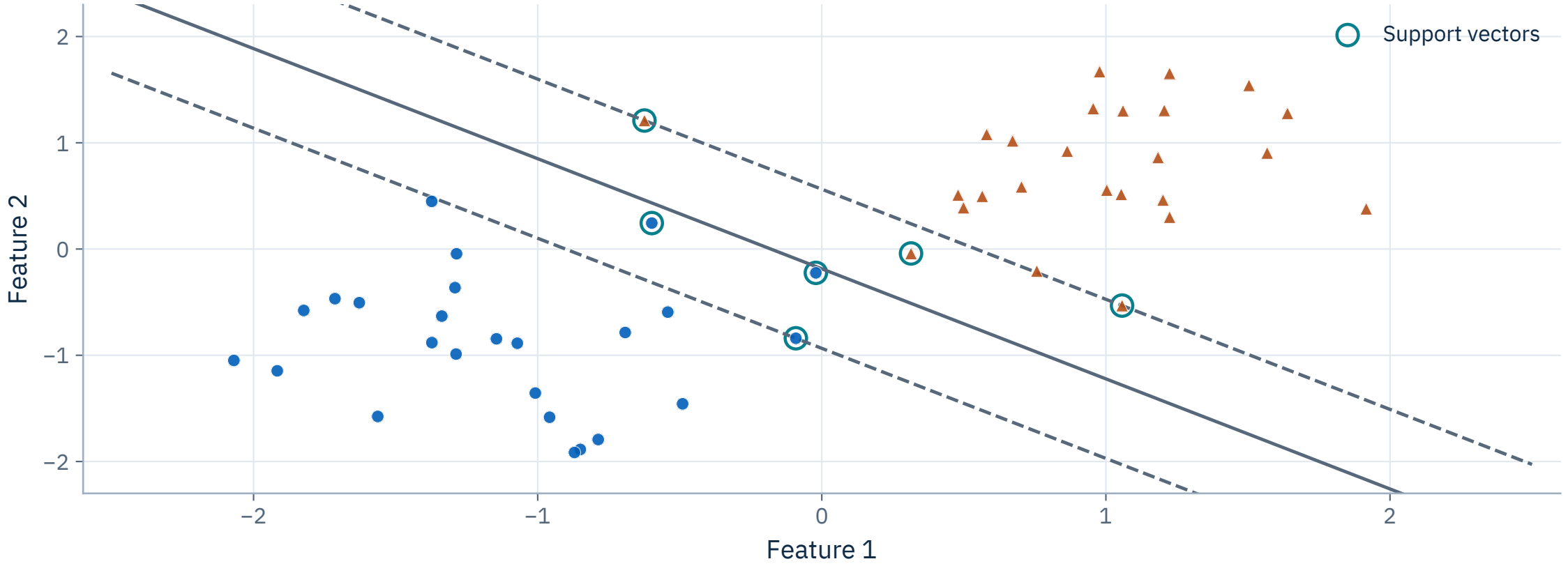
- $\sum_j |a_j - b_j|$
- One large difference counts less

Cosine distance

- $1 - \cos\theta$, θ the angle between the rows
- Compares direction, not magnitude

◆ In many dimensions, nearest and farthest points become harder to tell apart (Module 4).

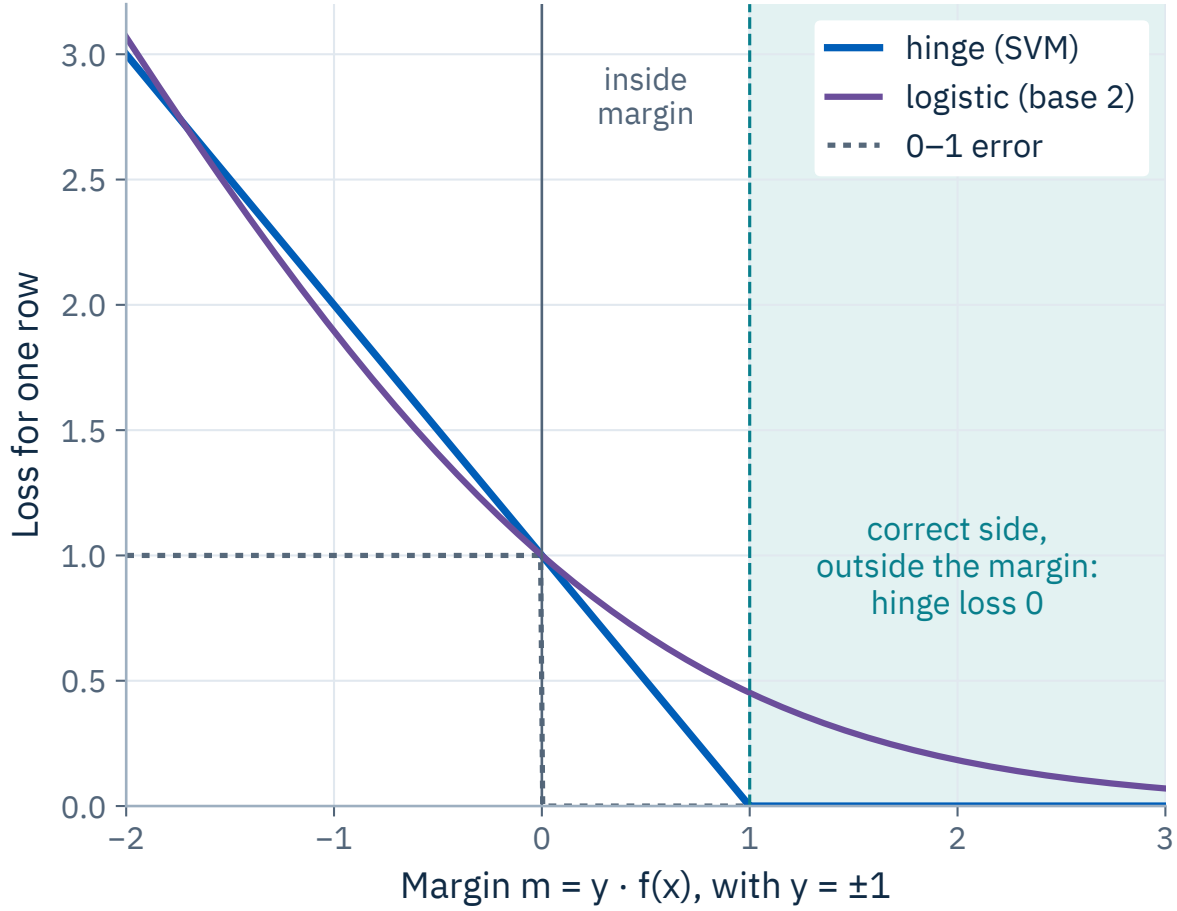
SVM separates classes with a margin



Linear SVM fitted to synthetic data. Dashed lines mark the margins; circled observations are support vectors.

Hinge loss ignores points beyond the margin

- Margin m = y·f(x), with labels y = −1 or +1
- **Hinge loss** max(0, 1 − m) is zero once m ≥ 1
- Only rows on or inside the margin shape the fit: the support vectors
- Logistic loss never reaches zero, so every row contributes



Exact loss functions of the margin m = y·f(x).

The C parameter trades margin for violations

$$\min_{w,\,b} \frac{1}{2}\|w\|^2 + C\sum_i \max(0,\, 1 - y_i f(x_i))$$

Small C

- Violations are cheap: wider margin
- More support vectors, smoother boundary

vs

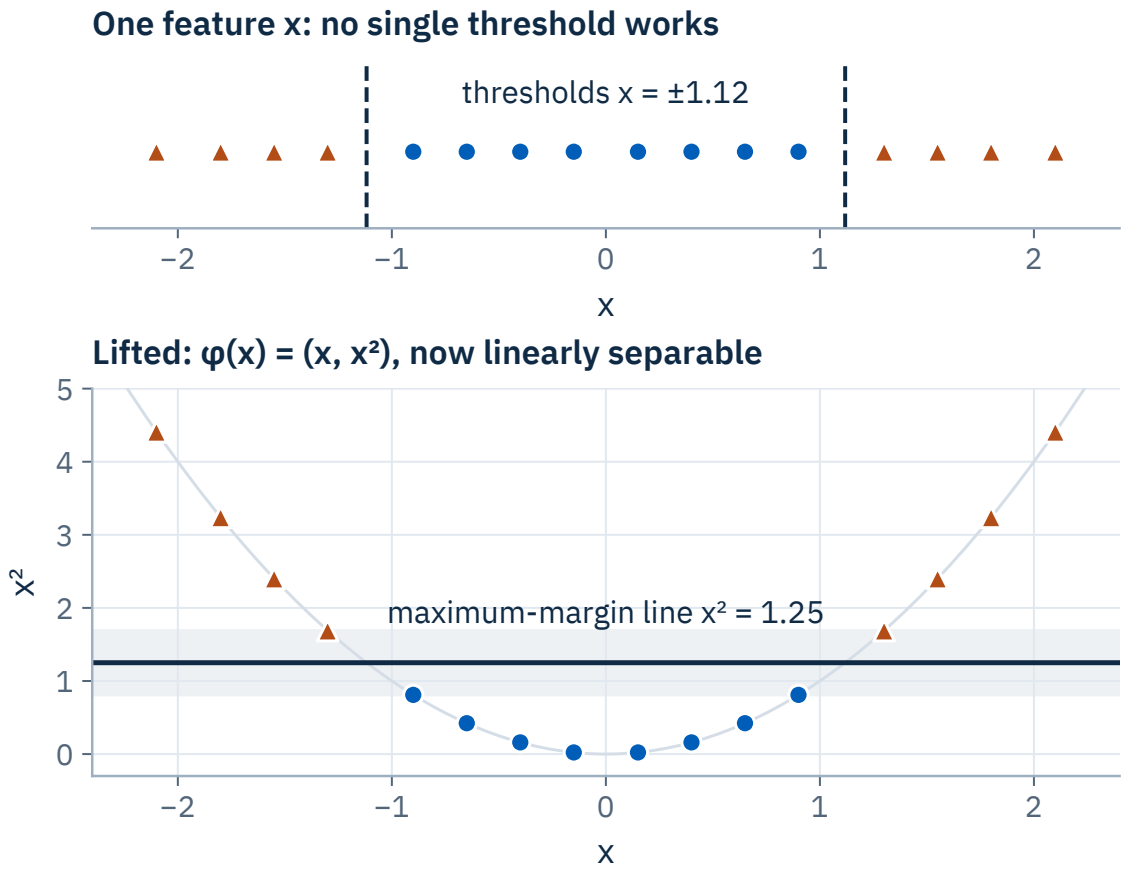
Large C

- Violations are expensive: narrower margin
- Chases hard training rows; can fit noise

◆ Search C and γ together after scaling; SVM scores are not probabilities by default.

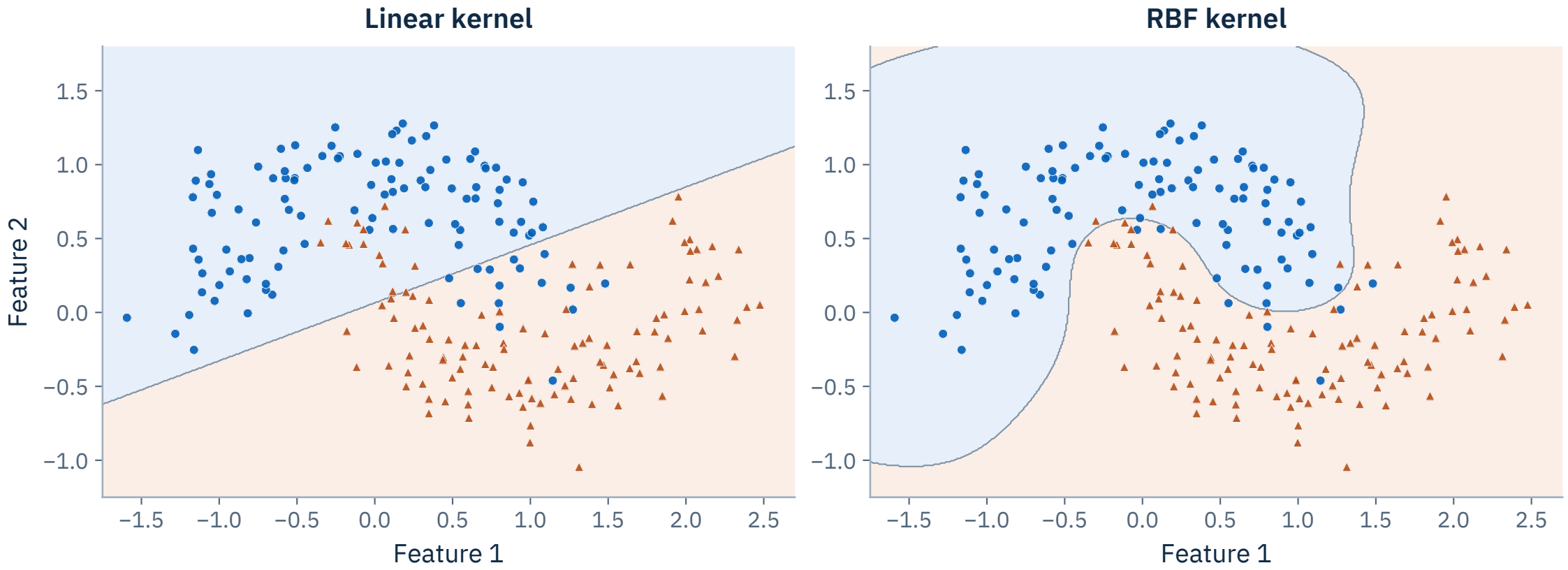
The kernel trick lifts data into a separable space

- One feature, class 0 in the middle: no single threshold works
- Map x to (x, x²): a straight line now separates the classes
- Back in x, that line becomes two thresholds at ±1.12
- The **kernel trick** computes such similarities without building φ(x)



Symmetric toy data, 8 points per class; maximum-margin line x² = 1.25 in the lifted space.

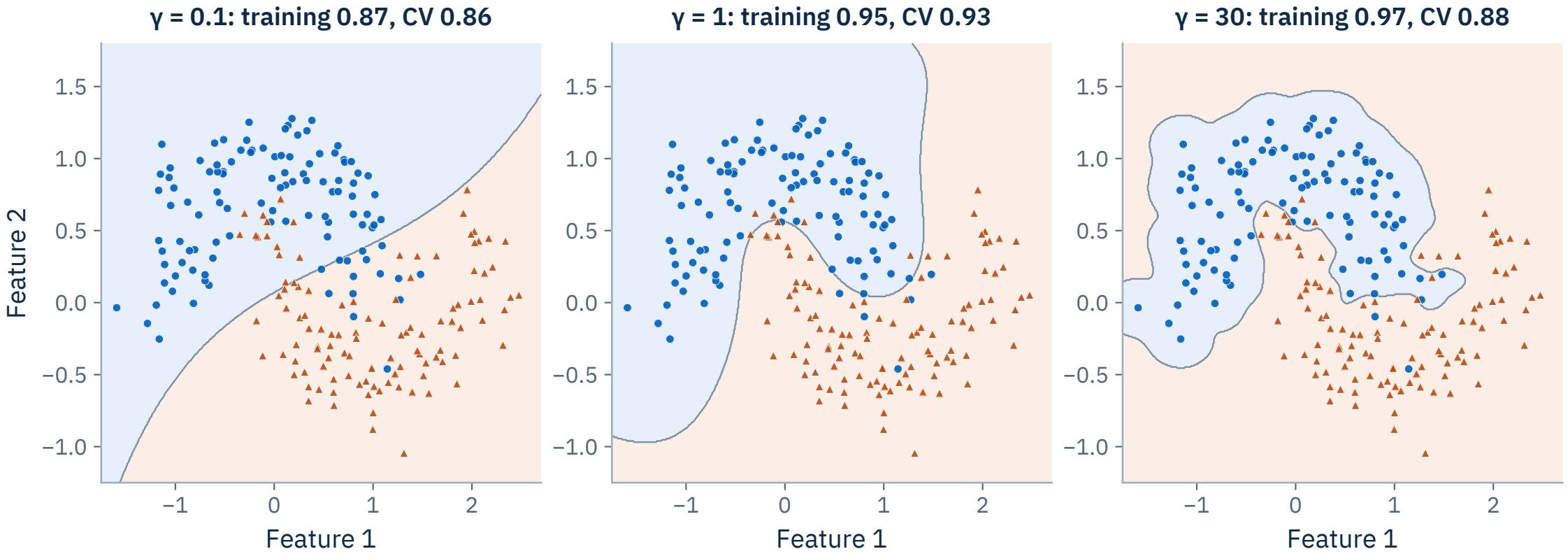
Kernels change the space of boundaries



Linear and RBF SVM fits on the same synthetic observations. The RBF model can represent a curved boundary.

Gamma sets how far each example's influence reaches

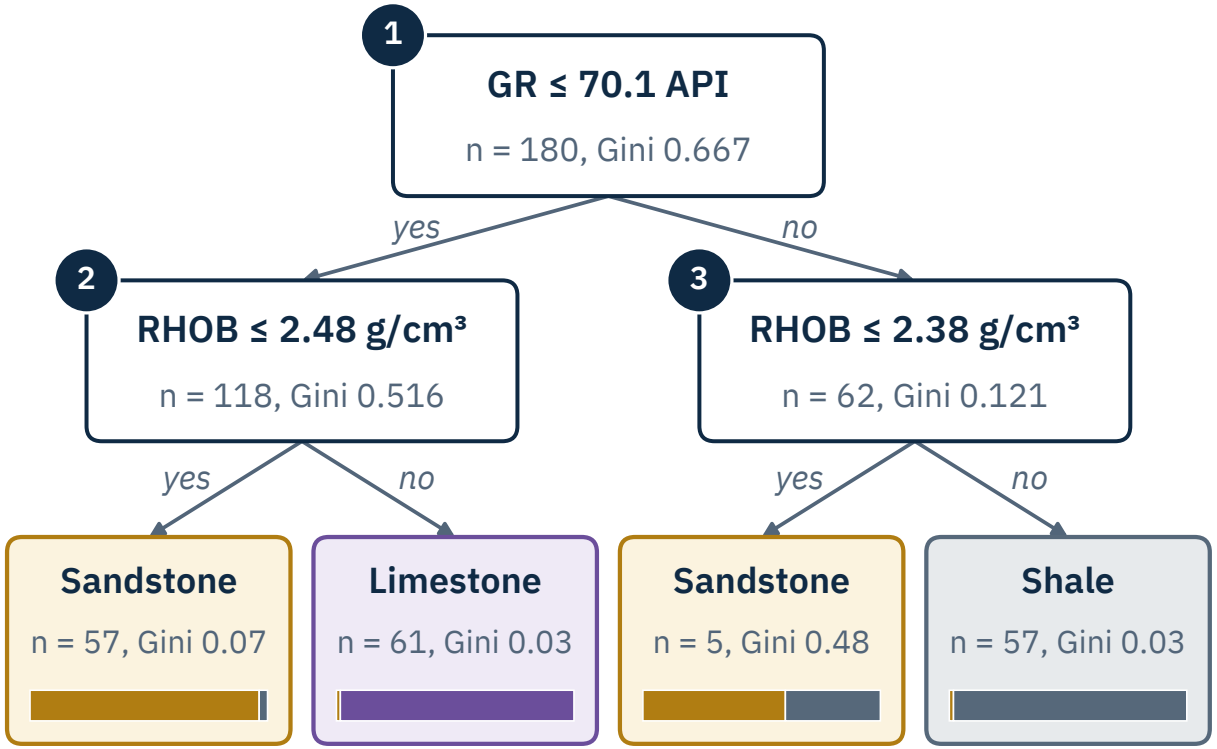
RBF kernel exp(−γ × squared distance): larger γ gives each training row a more local influence.

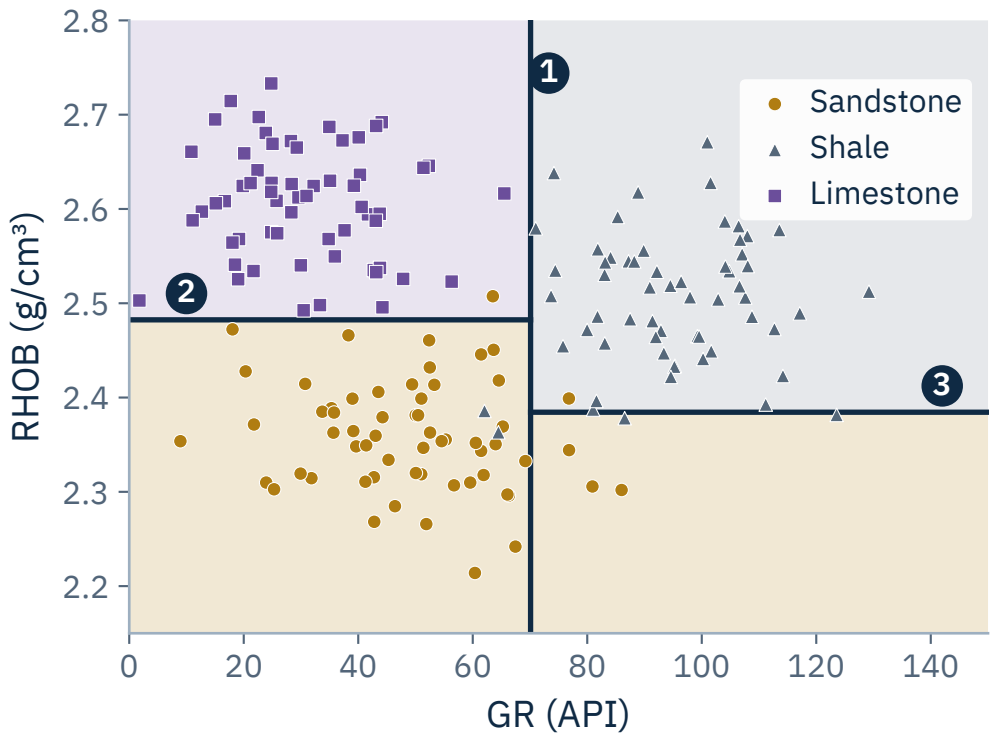


Same synthetic moons as the shared figures (seed 42); standardized RBF SVM with C = 1; accuracy on training rows and 5-fold stratified CV.

Trees ask a sequence of threshold questions

Each internal node tests one feature against a threshold; each leaf predicts its training majority.



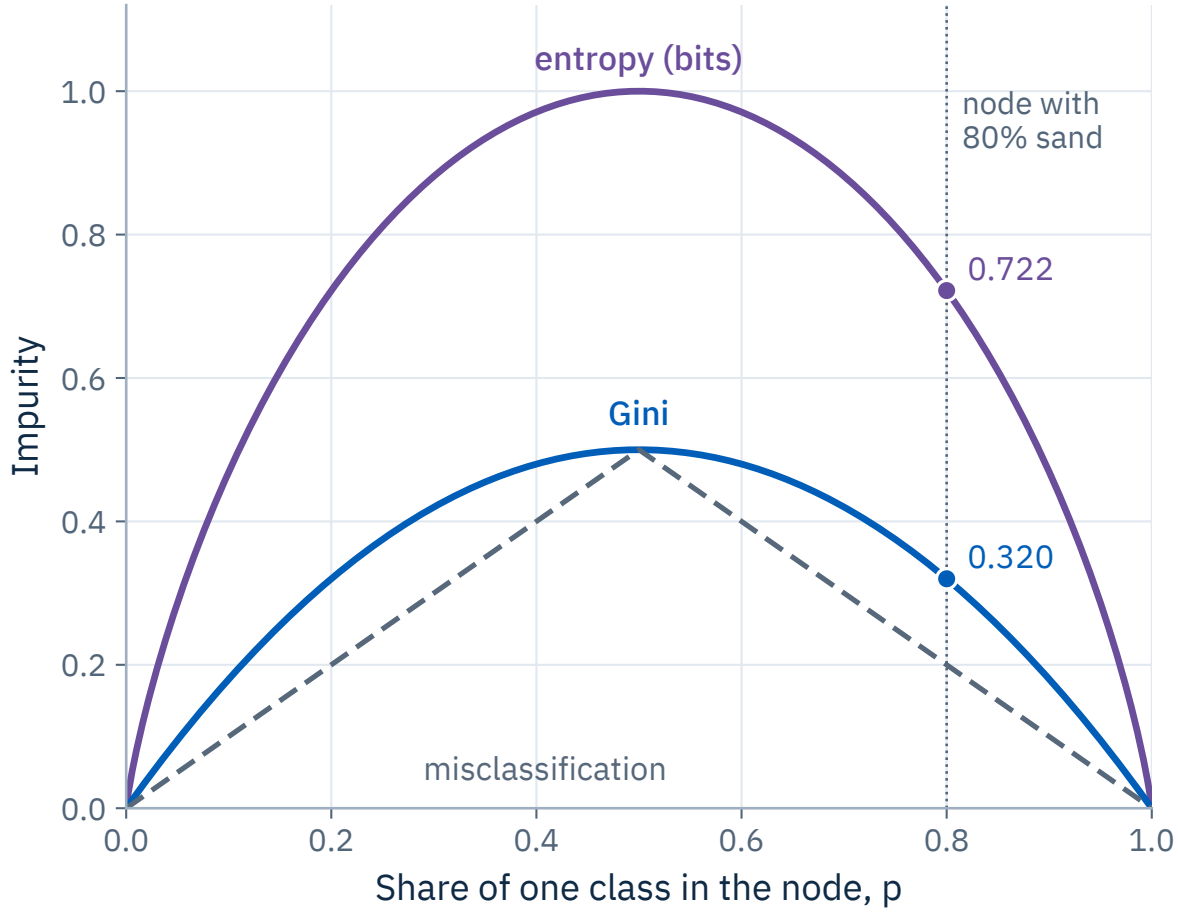


Synthetic facies samples (60 per class, seed 2); a depth-2 tree drawn from scikit-learn's fitted tree_ arrays.

Impurity measures how mixed a node is

$$\text{Gini} = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k$$

- Both are zero for a pure node and largest at 50/50
- A split is scored by the drop in size-weighted child **impurity**
- Gini and entropy usually choose similar splits



Exact two-class impurity functions; marked values at p = 0.8.

PRACTICE

5 min · Pairs

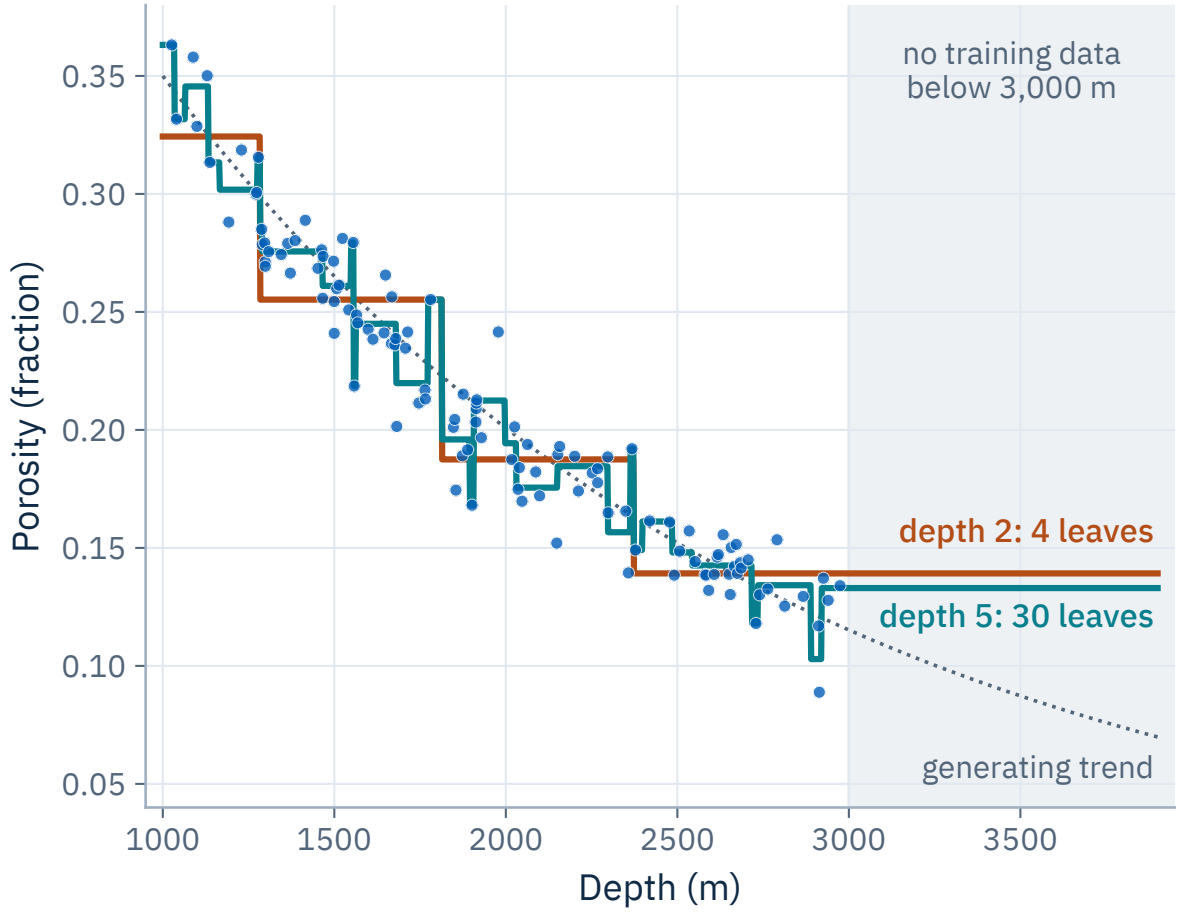
Choose the better first split

Candidate split	Left child: sand / shale	Right child: sand / shale
A: GR ≤ 60 API	8 / 2	2 / 8
B: RHOB ≤ 2.40 g/cm³	9 / 5	1 / 5

1. The parent holds 10 sand and 10 shale: compute its Gini impurity.
2. Compute each child's Gini, weight by child size, and pick the split.

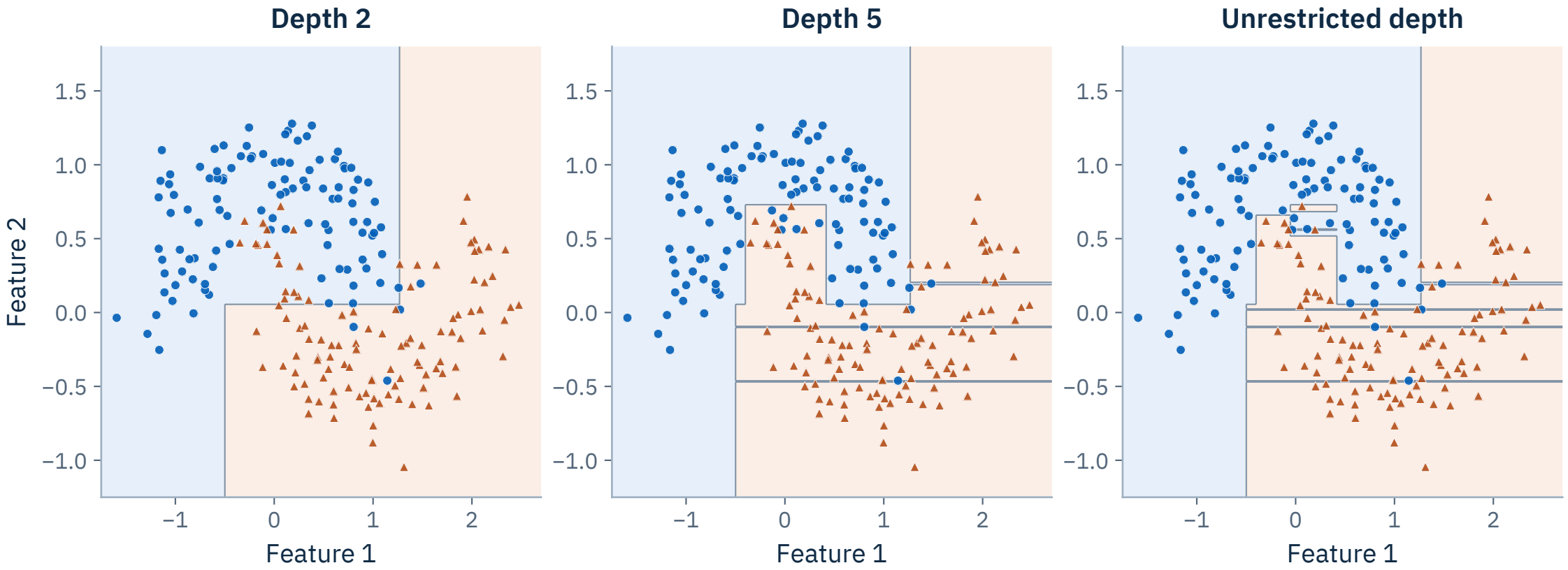
Regression trees predict a constant in each leaf

- Each leaf predicts the mean target of its training rows
- Splits are chosen to reduce the squared error
- The fit is a staircase and cannot extrapolate beyond the data
- Depth and min_samples_leaf set the number of steps



Synthetic compaction trend (seed 10), 120 samples; trees of depth 2 and 5.

Depth controls tree complexity



Trees fitted to the same synthetic observations. More depth permits smaller regions and finer partitions of the training data.

LABORATORY

45 min

Lab 2: classifiers and a facies split design

- ☐ §3: KNN on Iris; plot training and CV accuracy against K
- ☐ §4: trees of depth 1, 2, 3, 5 and unlimited; lock one model, test it once
- ☐ Facies notebook §2–4: audit missing and implausible values by well
- ☐ Facies notebook §6 and §9: write a well-grouped evaluation and tuning plan

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Accuracy against K and depth, with a majority-class baseline
- A facies data-audit table by well
- A split and tuning plan for unseen wells

DISCUSSION

Lab 2 debrief: the facies file needs a well-aware plan

Computed from facies_dataset.csv: 4,149 rows from 10 wells.

22%

PE missing; 100% in two wells

12

PE values of 326–328.5 (median 3.6)

25

GR values above 200 API

10

wells: the real sample size for CV

◆ Same forest, same nine wells: macro F1 0.76 with shuffled row folds, 0.38 with well folds.

PART 6 OF 6

06

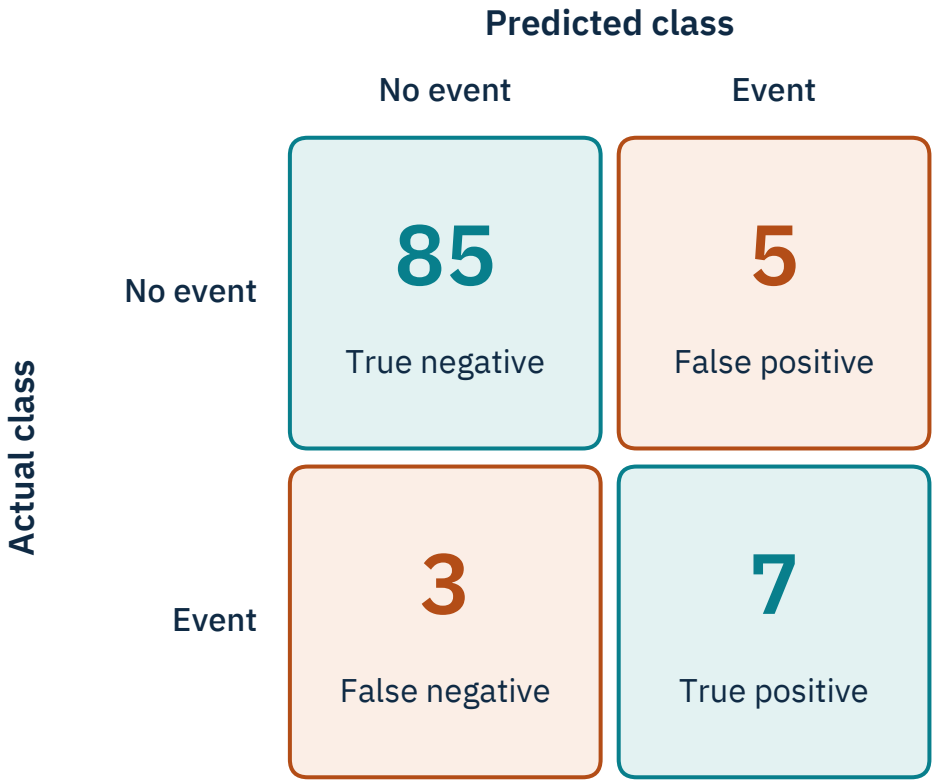
Evaluating and selecting models

A score is only as good as the experiment behind it.

- Confusion matrices and metric choice
- ROC curves and thresholds
- Multi-class averages
- Pipelines, grouped tuning and fair comparison

A confusion matrix preserves the counts

- Rows are actual classes; columns are predictions
- False positives are false alarms; false negatives are missed events
- State which class is the event before quoting any metric
- Some tools transpose this layout: check before reading



Illustrative counts: 100 observations, 10 actual events. Not model performance.

PRACTICE

4 min · Pairs

Accuracy can hide the minority class

Using TN = 85, FP = 5, FN = 3, TP = 7:

1. Calculate accuracy, event precision and event recall.
2. Compare with a model that always predicts no event.
3. Which error would matter most for the proposed use?

DISCUSSION

Never alerting still scores 90% accuracy

Same 100 observations, 10 real events:

92%

accuracy of the model

58%

event precision: 7 of 12 alerts

70%

event recall: 7 of 10 events

90%

accuracy of never alerting

- Report per-class precision and recall whenever events are rare.

Metric choice follows the decision

Precision

- How often an alert is right
- TP / (TP + FP)

Recall

- Share of real events caught
- TP / (TP + FN)

F1

- Harmonic mean of the two
- Encodes no business costs

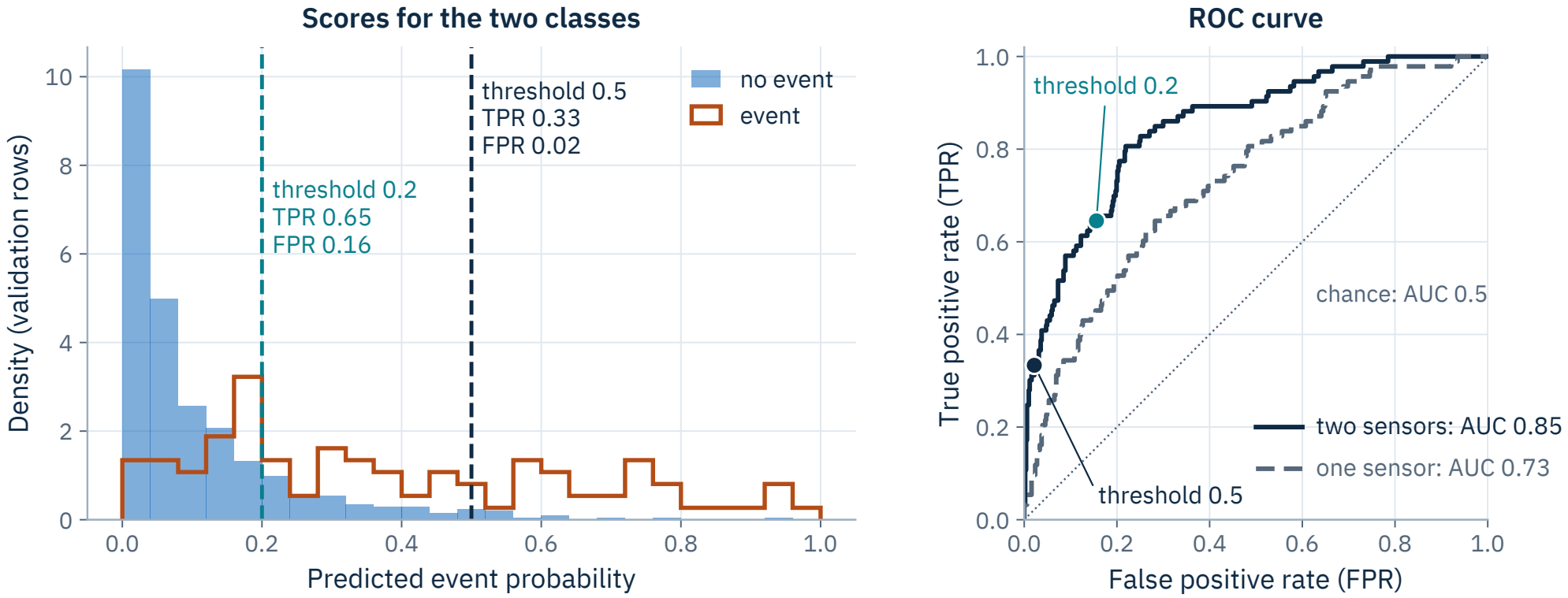
Macro average

- Mean over classes
- Rare facies count as much as common ones

◆ Agree the metric with the decision owner before looking at any scores.

ROC curves summarize every threshold at once

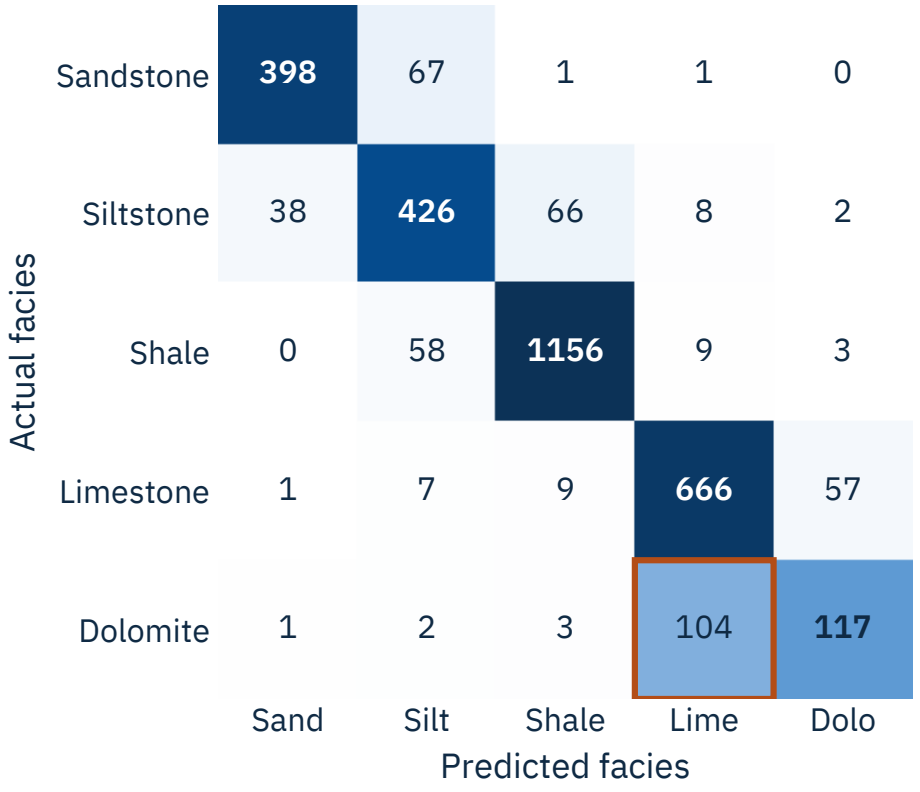
Moving the threshold trades false alarms for missed events; **AUC** summarizes the ranking.

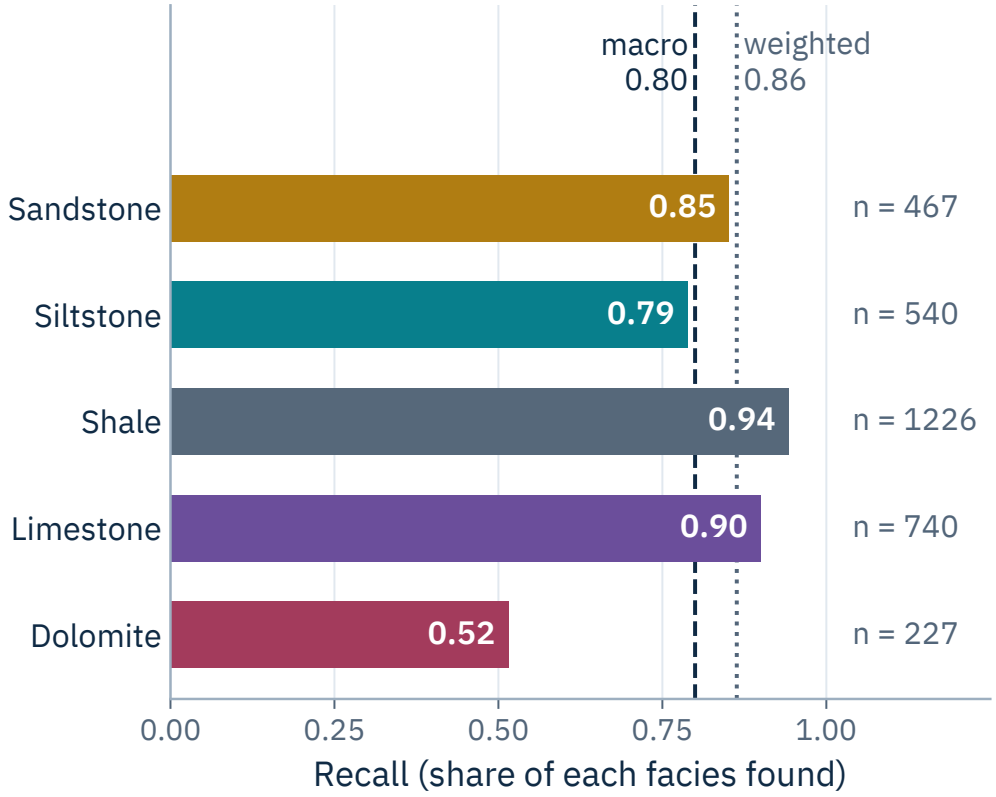


Synthetic ESP-trip data, 15% events (seed 13); logistic regression trained on 1,000 rows, evaluated on 600 validation rows. Precision–recall curves for rare events: Module 5.

Multi-class errors hide off the diagonal

Dolomite is found only about half the time; most of its misses are called limestone.





Synthetic 8-well facies data (seed 7); standardized RBF SVM, leave-one-well-out predictions. Colour = share of each actual facies.

Macro and weighted averages answer different questions

Recall on the synthetic facies example, from the same leave-one-well-out predictions:

0.52–0.94

per facies: one value per class

0.80

macro: every facies counts equally

0.86

weighted by frequency: equals accuracy

◆ A rare facies found poorly drags the macro average down; the weighted average hides it.

A pipeline keeps training steps together

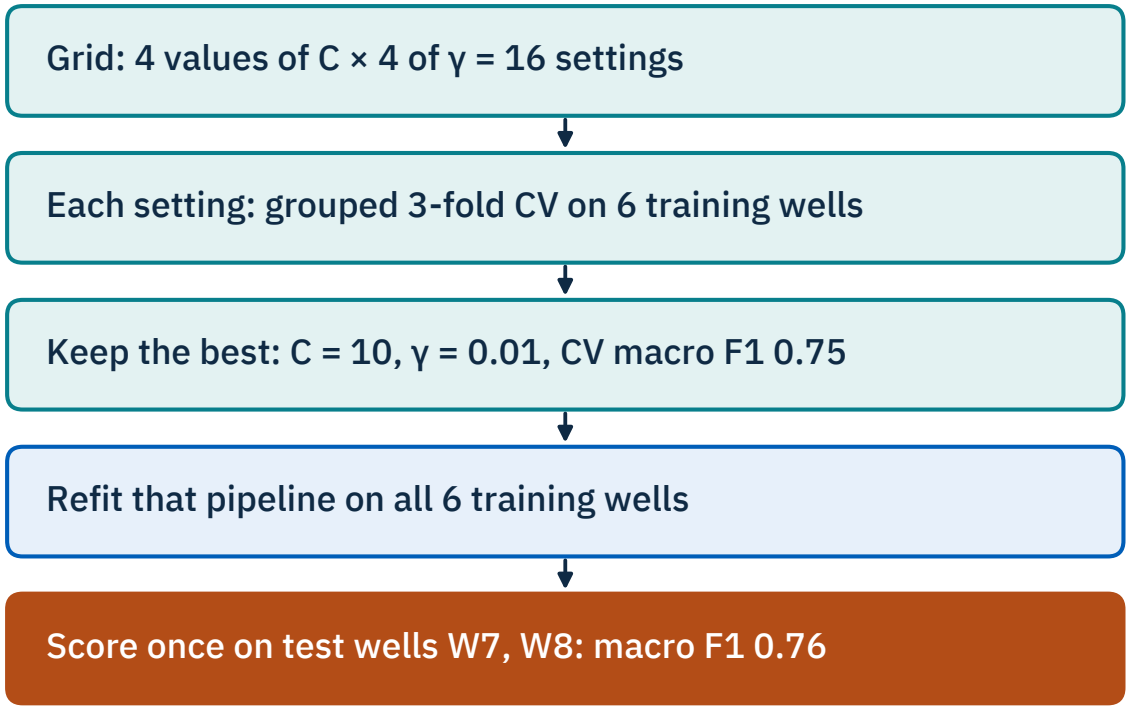
```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

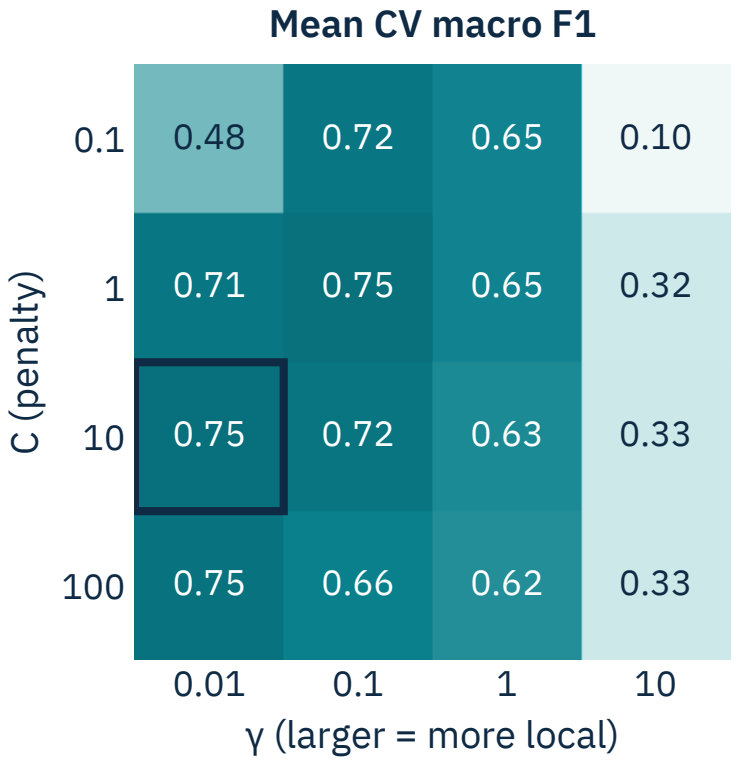
model = make_pipeline(
    SimpleImputer(strategy="median", add_indicator=True),
    StandardScaler(),
    SVC(kernel="rbf"),
)
```

Every cross-validation fit learns fresh imputation and scaling statistics.

Grid search scores every setting with grouped CV

16 settings × 3 grouped folds = 48 fits and one refit; the test wells are opened once.





Synthetic 8-well facies data (seed 7): GridSearchCV with GroupKFold(3) on wells W1–W6; W7–W8 as the test set.

Grouped tuning respects well boundaries

```python
from sklearn.model_selection import GroupKFold, GridSearchCV

search = GridSearchCV(
    model,
    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]},
    cv=GroupKFold(n_splits=3), scoring="f1_macro",
)
search.fit(X_train, y_train, groups=well_train)
print(search.best_params_, round(search.best_score_, 2))
```

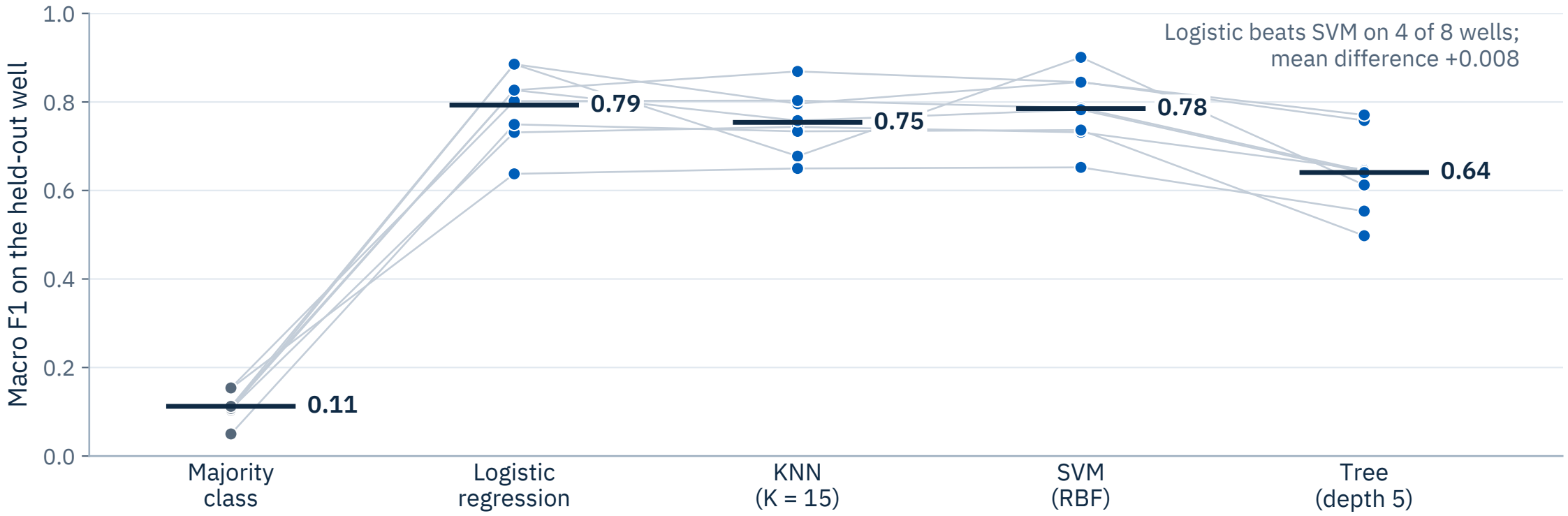
OUTPUT

```
{'svc__C': 10, 'svc__gamma': 0.01} 0.75
```

Run on training wells W1–W6 of the synthetic data; W7 and W8 stay closed.

Compare models by their fold-score distributions

Same eight held-out wells for every pipeline: read the spread and the paired lines.



Synthetic 8-well facies data (seed 7); leave-one-well-out macro F1 for five pipelines on identical folds. Grey lines join scores of one well.

Model comparison is a controlled experiment

Candidate	Strong starting point when	Main check
Linear or logistic	Effects are roughly additive	Residual or probability structure
KNN	Local similarity is meaningful	Scaling and prediction cost
SVM	A margin or kernel helps	C, γ and calibration
Tree	Threshold interactions are plausible	Leaf size and stability

Hold the split, folds and metric fixed; the baseline row comes first.

◆ Pair every score with an error review and the runtime at the point of use.

LABORATORY

25 min

Lab 3: a tuned pipeline and a locked test

- §1: explain why PCA fitted on all rows leaks information
- §2–3: fit the 2-D teaching models and record the retained variance
- §4: run the CV grid search; record the kernel, C and γ it selects
- §5: add PCA inside the pipeline; compare 2, 10 and 30 components
- §6: set RUN_FINAL = True once, after choosing, and report that score

NOTEBOOK

`exercises/03_svm/SVM_Exercise_Student.ipynb`

EVIDENCE TO SUBMIT

- What each partition was used for
- A matched comparison table
- One final score with its caveat

DISCUSSION

Lab 3 debrief: small holdouts give noisy verdicts

Reference run of the SVM notebook on the local Sonar file (208 rows):

124/42/42

training, validation and final rows

39%

variance kept by 2 principal components

0.79

CV balanced accuracy of the chosen RBF SVM

2.4

accuracy points per row in a 42-row holdout

- The same model scored 0.92 on the 42 validation rows: one procedure, two noisy numbers.

Key takeaways

- ✓ Write the prediction contract before touching a model.
- ✓ Audit the data: null codes, units, depth shifts and patterned gaps.
- ✓ Make the split answer the deployment question: group by well, respect time.
- ✓ Fit every preprocessing step inside the training fold of a pipeline.
- ✓ Compare regularized candidates with a baseline on identical grouped folds.
- ✓ Report per-class metrics and all fold scores; open the test set once.

PRACTICE

5 min | Individual

Exit check: four questions before you leave

1. Why must scaling run inside a cross-validation pipeline?
2. When is a random row split misleading for well logs?
3. A shale sample was predicted at p = 0.1. What is its log loss, and why is it large?
4. What evidence would justify a more complex model?

References and further study

- scikit-learn User Guide: linear models, neighbours, SVM, trees, cross-validation and common pitfalls.
- James, Witten, Hastie, Tibshirani: *An Introduction to Statistical Learning*; Hastie, Tibshirani, Friedman: *The Elements of Statistical Learning*.
- Kaufman, Rosset, Perlich, Stitelman (2012). Leakage in data mining: formulation, detection, and avoidance. *ACM TKDD* 6(4).
- Roberts et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography* 40.
- Hall (2016). Facies classification using machine learning. *The Leading Edge* 35(10).